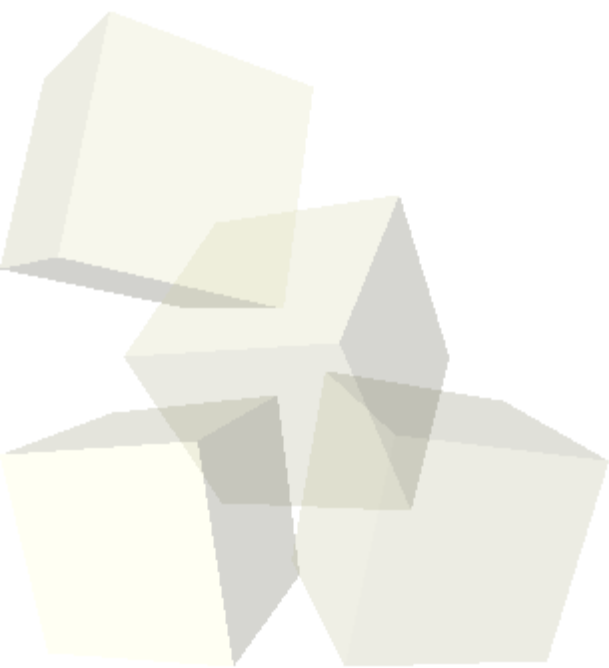




VIRTUALIZACE V LINUXU



Honza Vrbata
vrbata@gopas.cz



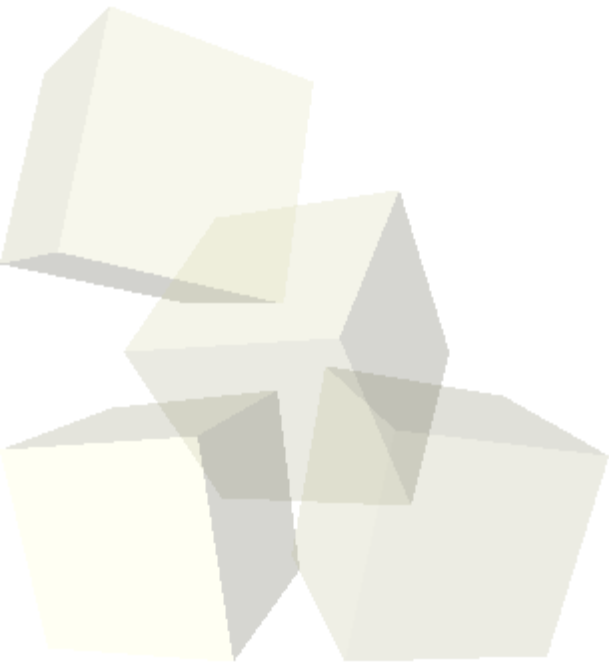
- Původní význam termínu virtualizace, pocházející z 60. let 20. století a znamená **vytváření virtuálních strojů za pomoci kombinace hardwaru a softwaru**. Pojem virtuální stroj zjevně pochází od pokusného stránkovacího mechanismu systému IBM M44/44X. Zakládání a správa virtuálních strojů byla v počátcích CP-40 také označována jako zakládání a správa pseudostrojů a později jako virtualizace serverů. Od té doby získal termín virtualizace a virtuální stroj další významy.
- **Virtualizace je prováděna na dané hardwarové platformě pomocí softwaru hostitele (řídící program), který vytváří simulované prostředí počítače (virtuální stroj) pro hostovaný software**. Software hosta, což často bývá celý operační systém, běží, jako by byl nainstalován na samostatné hardwarové platformě. Typicky je simulováno více takových virtuálních strojů na jednom fyzickém stroji. Pro správnou funkci hosta je třeba, aby simulace byla dostatečně robustní, aby podporovala všechna vnější rozhraní hostovaného systému, což (vzhledem k druhu virtualizace) může zahrnovat ovladače hardwaru.
- **Existuje řada přístupů k virtualizaci.**

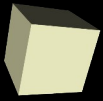


■ Emulace nebo Simulace :

Virtuální stroj simuluje celý hardware, dovoluje běh neupraveného OS hosta i na zcela odlišném procesoru. Tento přístup je dlouho používán například za účelem tvorby softwaru pro procesory, které nejsou fyzicky dostupné.

Příklady : Bochs, PearPC, **Qemu**, atd.

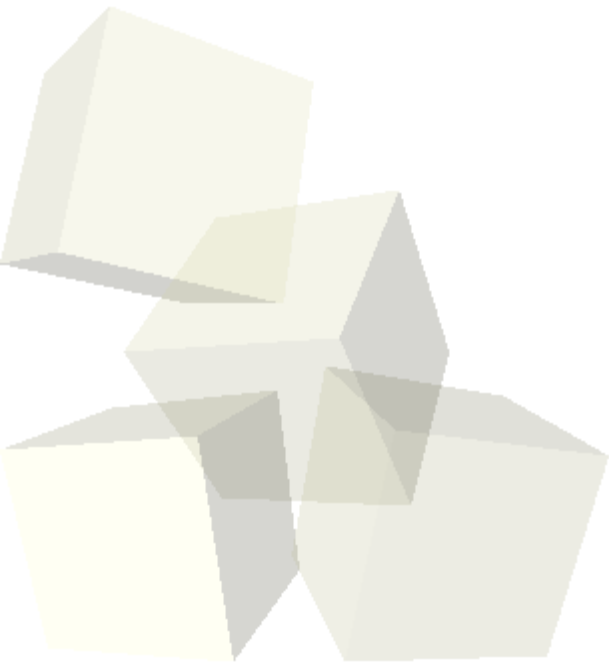




■ Nativní a plná virtualizace :

Virtuální stroj simuluje dostatečné množství hardwaru tak, aby umožnil oddělený běh **neupraveného** OS hosta určeného pro stejný druh CPU. Obvykle je možný souběh více instancí.

Příklady : VirtualBox, VirtualPC, Virtuallron, Parallel Desktop, VMWare Workstation, VMWare Server, Win4Lin, **KVM**, **XEN**, atd.

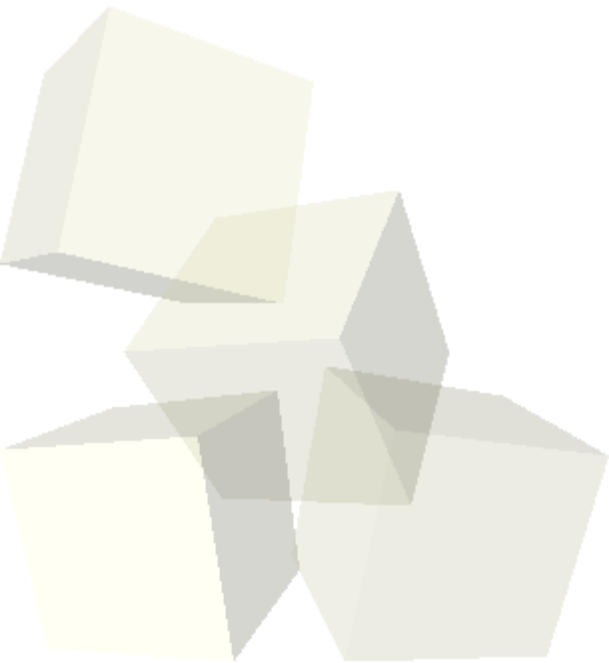




■ Paravirtualizace :

Virtuální stroj nemusí nezbytně simulovat hardware, ale místo toho (nebo navíc) nabízí zvláštní API, které může být použito jen z **upraveného** hostovaného OS. Toto systémové volání **hypervizora** se nazývá „hypercall“.

Příklady : **XEN**, **KVM**, atd.

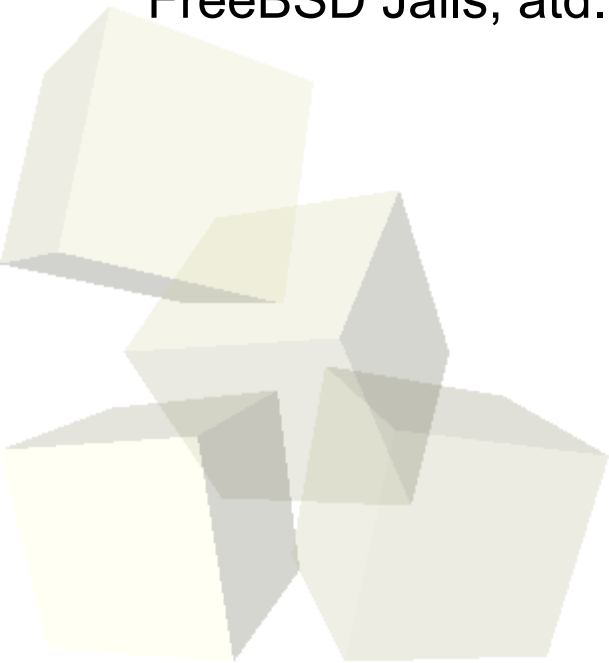




■ Virtualizace na úrovni operačního systému (*partitioning*, **containers**) :

Virtualizuje se fyzický server na úrovni OS, což umožňuje běh více izolovaných bezpečných virtuálních serverů na jednom fyzickém serveru. Prostředí hostovaného OS sdílí jeden OS s hostitelským systémem – tj. stejné jádro OS je použito pro implementaci hostovaného OS. Aplikace běžící v hostovaném prostředí jej však vnímají jako samostatný systém.

Příklady : LINUX VServer, **OpenVZ** (Virtuozzo), **LXC**, **LXD**, **Incus**, **Docker**, FreeBSD Jails, atd.





■ Aplikační virtualizace :

Desktopové nebo serverové aplikace běžící na daném stroji, používají místní zdroje, ale běží ve zvláštním virtuálním stroji. To je rozdíl oproti tradičnímu lokálnímu běhu nativních aplikací, tj. softwaru nainstalovanému přímo na systému. Taková aplikace běží v malém virtuálním prostředí obsahujícím komponenty nutné ke spuštění – např. položky registrů, soubory, proměnné prostředí, prvky uživatelského rozhraní a globální objekty. Toto virtuální prostředí se chová jako vrstva mezi aplikací a operačním systémem, která zabraňuje konfliktům mezi aplikací a OS nebo mezi aplikacemi vzájemně.

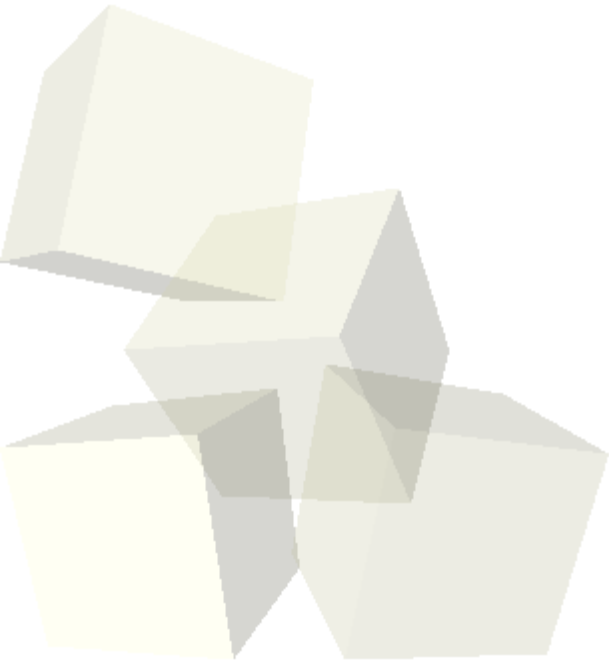
Příklady : **JAVA** Virtual Machine, **.NET** Framework, atd.





OpenVZ

<http://openvz.org/>





■ Co je OpenVZ ?

- ♦ **Kontejnerové** řešení pro LINUX.
- ♦ Umožňuje vytvářet **bezpečná** a **oddělená** virtuální prostředí **VE** (virtual environment). V těchto VE běží jednotlivé, separované "instance" OS.
- ♦ Ve VE je možné limitovat systémové a další zdroje.
- ♦ Jednotlivé VE se navzájem neovlivňují.

■ Podobná řešení :

- ♦ **LINUX VServer**, **LXC**, **LXD**, FreeBSD Jail, Solaris Containers, ...



■ Možnosti využití a další vlastnosti :

- ♦ Virtuální síťové servery poskytovatelů služeb.
- ♦ Provoz jednotlivých démonů (www, mail, databáze) v izolovaných **VE** kvůli **bezpečnosti**.
- ♦ Lepší využití zdrojů daného počítače díky možnosti provozování více oddělených **VE** - **konsolidace**.
- ♦ Stoprocentní **dostupnost** díky migraci **VE** za běhu.
- ♦ Na jednom stroji je možné provozovat **desítky i stovky** jednotlivých **VE** !

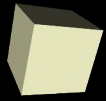
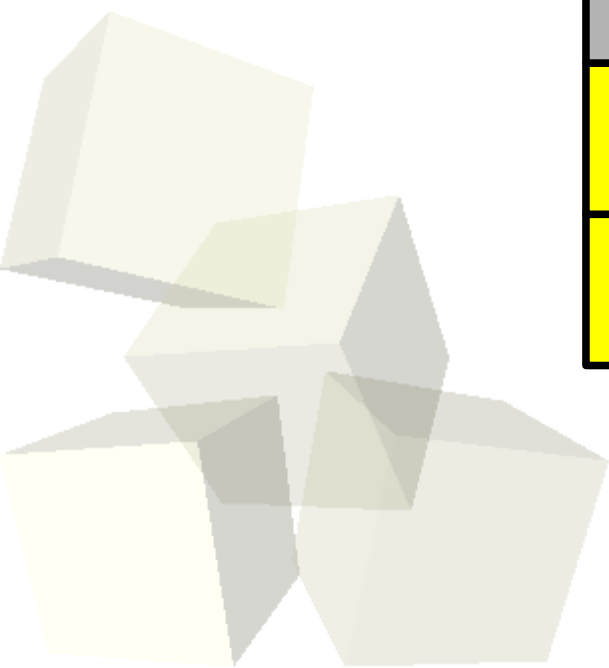
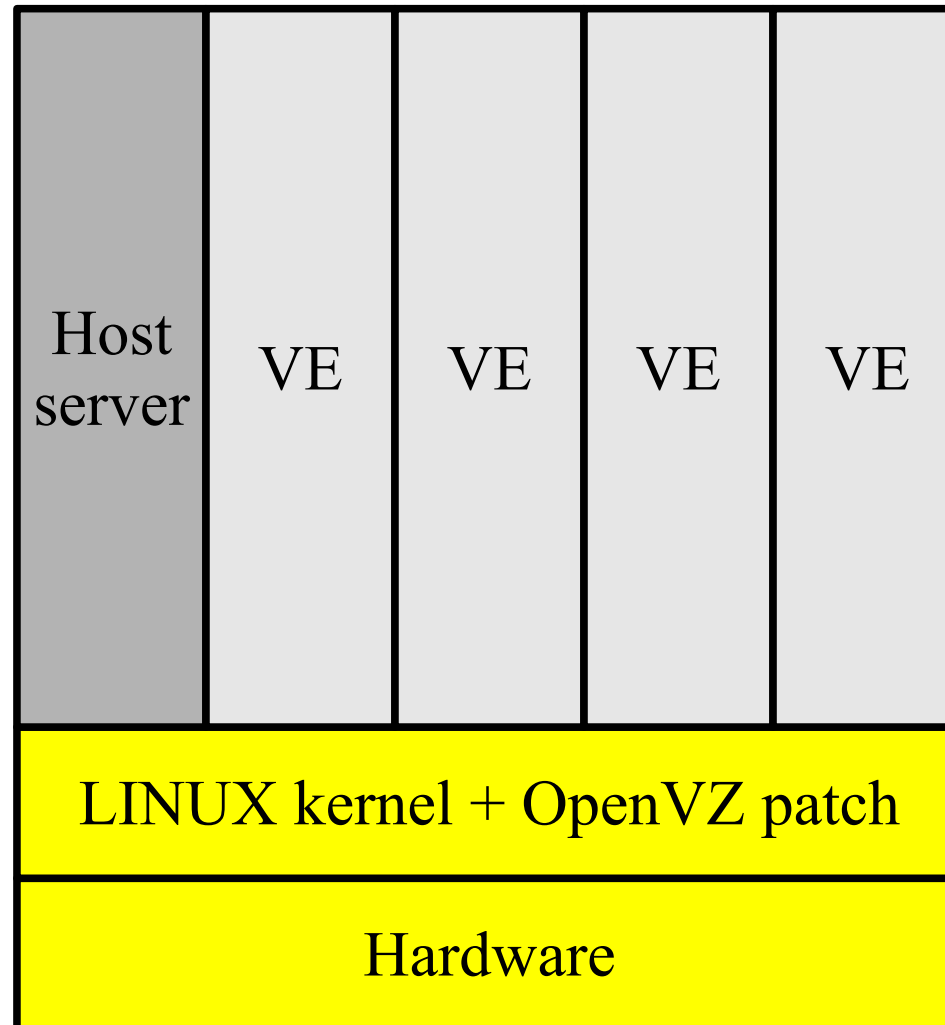


Schéma :





■ Každé VE je samostatnou jednotkou a disponuje vlastními :

- ♦ **soubory**

- systémové knihovny
- soubory aplikací
- virtualizované adresáře */dev*, */proc* a */sys*
- zámky, atd.

- ♦ **uživatelé a skupiny**

- ♦ **procesy**

- každé VE má vlastní strom procesů počínaje procesem init s PID=1

- ♦ **síťové prostředky**

- každé VE může disponovat vlastním virtuálním síťovým rozhraním s vlastní IP adresou, atd.

- ♦ **HW zařízení**

- jednotlivé VE může mít privilegovaný přístup k síťovému rozhraní, oblasti na disku a dalším zařízením.

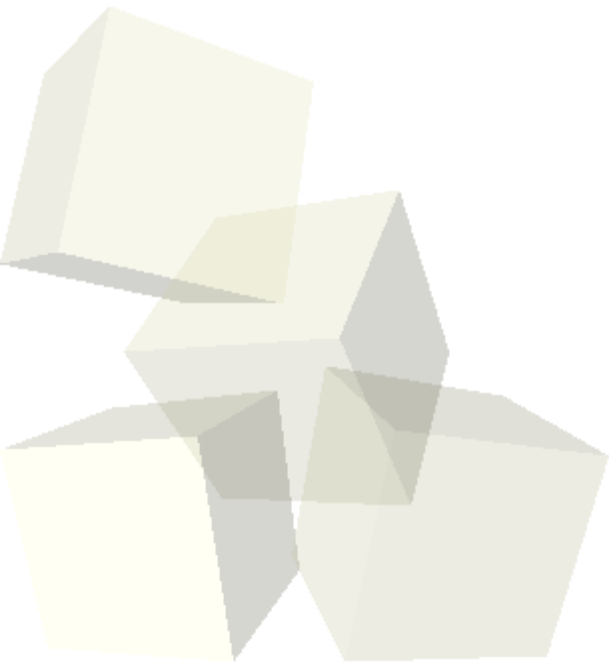


- ♦ ***IPC prostředky***

- sdílená paměť, semaforey, fronty zpráv, atd.

- **Další vlastnosti :**

- ♦ pokročilé nastavení (diskové kvóty, limitace CPU, paměti, ...)
 - ♦ migrace VE za běhu bez přerušení činnosti.
 - ♦





■ Instalace OpenVZ na OS typu RedHat :

- ♦ Standardní instalace RH. Je vhodné vytvořit nativní oblast s **ext2/ext3/ext4** připojenou do **/vz**.
- ♦ **Instalace OpenVZ jádra a dalších nástrojů :**
 - `cd /etc/yum.repos.d`
 - `wget http://download.openvz.org/openvz.repo`
 - `rpm --import http://download.openvz.org/RPM-GPG-Key-OpenVZ`
 - `yum install vzkernel`
 - `yum install vzctl vzquota ploop`
- ♦ **Deaktivace systému SELinux :**
 - `/etc/sysconfig/selinux`
 - `SELINUX=disabled`



■ Změny v nastavení *sysctl* kvůli síťování :

- ♦ */etc/sysctl.conf*

- `net.ipv4.ip_forward = 1`
`net.ipv4.conf.default.proxy_arp = 0`
`net.ipv4.conf.all.rp_filter = 1`
`kernel.sysrq = 1`
`net.ipv4.conf.default.send_redirects = 1`
`net.ipv4.conf.all.send_redirects = 0`
`net.ipv4.icmp_echo_ignore_broadcasts=1`
`net.ipv4.conf.default.forwarding=1`

■ *sysctl -p*

■ Nastavení parametrů jaderného modulu :

- ♦ */etc/modprobe.d/openvz.conf*

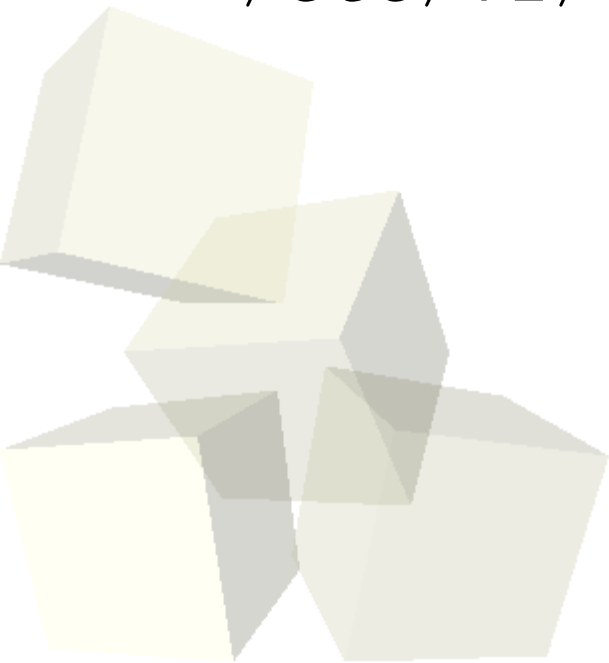


■ Inicializace OpenVZ démonů při startu :

- ♦ `/etc/rc.d/init.d/vz start`

■ Konfigurace :

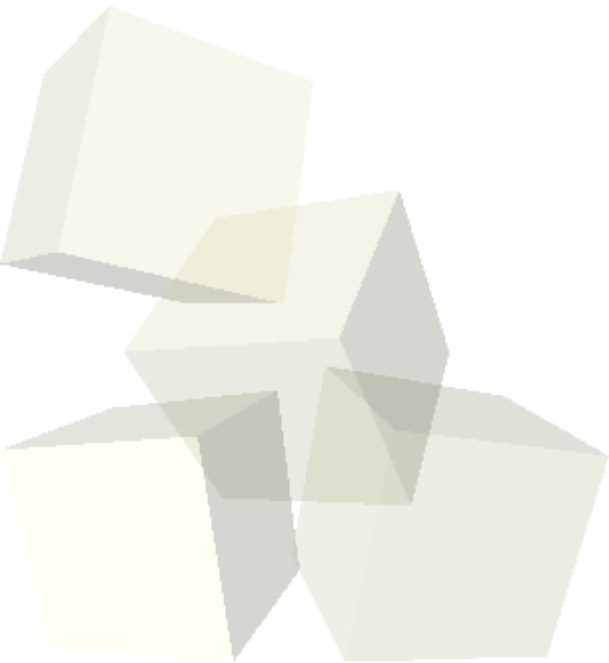
- ♦ `/etc/vz/`





■ Struktura adresáře **/vz** :

- ♦ dump
- lock
- private**
- root**
- template**
- vztmp





■ Instalace systému do VE :

- OpenVZ umožňuje vytvářet rychle jednotlivé instalace z připravených šablon (*šablony mohou být definovány pouze metadaty pro daný OS nebo existuje cache šablony, která obsahuje balíky a je zkomprimována do tgz formátu*).

→ <http://download.openvz.org/template/precreated/>

- Alternativně je možné nainstalovat systém pomocí nástrojů jako je **debootstrap**, atd.
- Je možné vytvářet vlastní šablony pro automatickou instalaci.



■ Instalace systému ze šablony :

- ♦ šablony jsou standardně umístěny v adresáři ***/vz/template/cache***
(*cache šablony v nerozbaleném **tgz** formátu*).

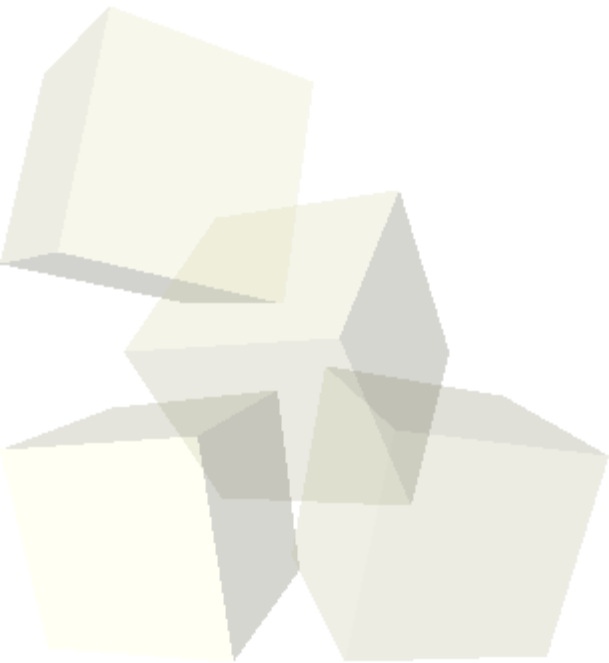
```
➤ vzctl create VE_ID --ostemplate centos-6-x86_64 --  
  config basic --layout [ploop,simfs]
```

- ♦ VE_ID je jednoznačný identifikátor VE (*hodnoty 0-100 jsou rezervovány pro vnitřní použití OpenVZ*).
- ♦ Parametr ***layout*** určuje jakým způsobem bude uložen souborový systém VE.



■ Souborový systém VE :

- ♦ **simfs** - tradiční způsob, soubory VE jsou umístěny v souborovém systému hostitelské systému v adresáři */vz/private/VEID/*
- ♦ **ploop** - souborový systém je umístěn v souborové image.





■ Souborový systém **ploop** :

- Konverze simfs na ploop :

→ **vzctl convert VEID**

- Změna velikosti (obousměrná, i za běhu) :

→ **vzctl set VEID --diskspace nnnG --save**





■ Ploop snapshots :

- **vzctl snapshot VEID** [--id \$UUID] [--name name] [--description desc] [--skip-suspend] [--skip-config]
- **vzctl snapshot-list VEID**
- **vzctl snapshot-mount VEID** --id uuid --target directory
- **vzctl snapshot-umount VEID** --id uuid
- **vzctl snapshot-switch VEID** --id uuid
- **vzctl snapshot-delete VEID** --id uuid



■ Základní nastavení parametrů :

- ♦ Síťové parametry :

- `vzctl set VE_ID --ipadd 10.3.0.x --nameserver 10.2.0.5 --hostname vz1 --save`

- ♦ VE_ID je jednoznačný identifikátor VE (*hodnoty 0-100 jsou rezervovány pro vnitřní použití OpenVZ*).

- ♦ Automatický start :

- `vzctl set VE_ID --onboot yes --save`



■ Ovládání systému ve VE :

- ♦ spuštění systému :
 - `vzctl start VE_ID`
- ♦ zastavení systému :
 - `vzctl stop VE_ID`
- ♦ restart systému :
 - `vzctl restart VE_ID`
- ♦ zobrazení informací o systému :
 - `vzctl status VE_ID`
- ♦ spuštění terminálu :
 - `vzctl enter VE_ID`
- ♦ spuštění příkazu v systému :
 - `vzctl exec VE_ID command`



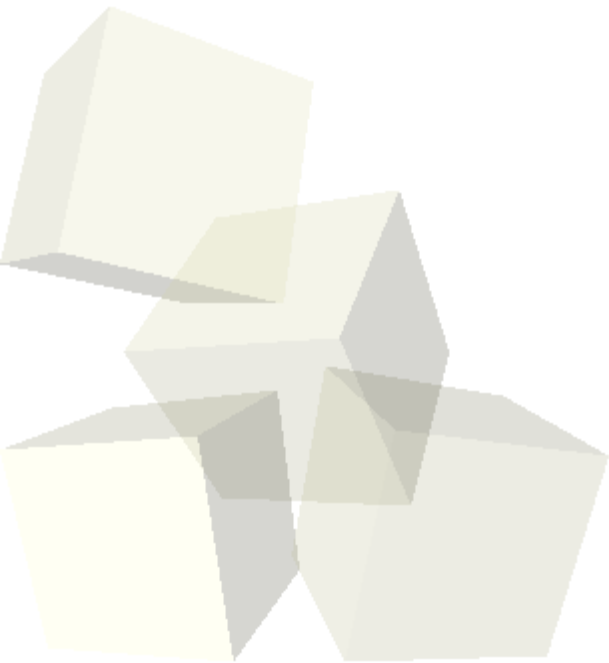
■ Checkpointing a obnovení systému :

- ♦ vytvoření obrazu běžícího VE :

→ `vzctl chkpnt VE_ID --dumpfile soubor`

- ♦ obnovení systému :

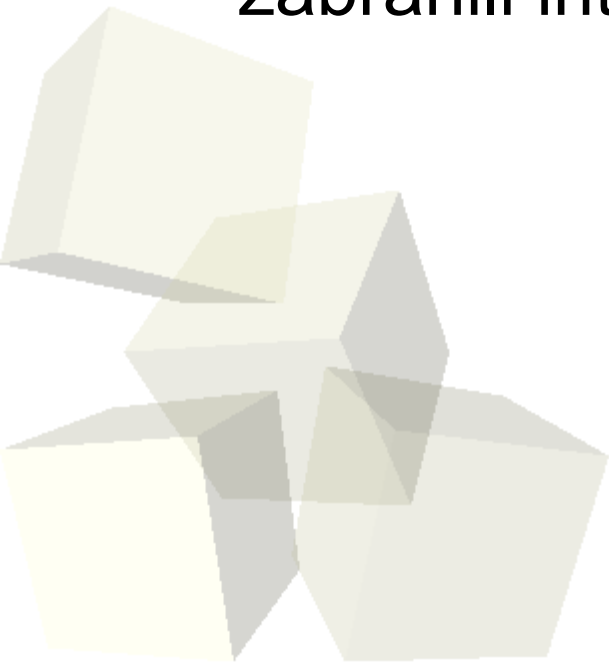
→ `vzctl restore VE_ID --dumpfile soubor`





■ Online migrate :

- ♦ vytvoření obrazu běžícího VE :
 - `vzmigrate --online host VE_ID`
- ♦ Pro přenos dat se používá protokol SCP (SSH). Je vhodné zajistit autorizaci pomocí klíčů, abychom zabránili interaktivnímu zadávání autorizačních údajů.





■ Sít'ová rozhraní :

- ♦ OpenVZ umožňuje ve VE vytvářet virtuální sít'ová rozhraní dvou druhů :
 - **venet** rozhraní pracuje pouze na sít'ové vrstvě, tedy IP.
 - **veth** rozhraní implementuje komplexní funkce sít'ového hardware, má vlastní MAC adresu a podporuje IPv6.





■ Síťová rozhraní *venet* :

- `vzctl set VE_ID --hostname hostname`
- `vzctl set VE_ID --ipadd addr`
- `vzctl set VE_ID --ipdel addr | all`
- `vzctl set VE_ID --nameserver addr`
- `vzctl set VE_ID --searchdomain domain`

- Řídící počítač se chová jako IP router (směrovač).



■ Síťová rozhraní *veth* :

- `vzctl set VE_ID --netif_add`
`<ifname>[,<mac>,<host_ifname>,<host_mac>,<bridge>]`

- **ifname** je jméno zařízení ve virtuálním systému.
- **mac** je mac adresa zařízení ve virtuálním systému.
- **host_ifname** je jméno zařízení v řídícím systému.
- **host_mac** je mac adresa zařízení v řídícím systému.
- **bridge** parametr umožňuje automaticky spustit skript pro přidání rozhraní do bridge.

```
vzctl set 101 --netif_add  
eth0,00:12:34:56:78:9A,veth101.0,00:12:34:56:78:9B
```



■ Nastavení bridge na hostitelském systému.

- Je vhodné deaktivovat **NetworkManager** a síť inicializovat „tradičním“ způsobem.
 - `chkconfig NetworkManager off`
 - `chkconfig network on`
 - `service NetworkManager stop`
- Příprava síťové karty **eth0** pro práci v bridge. Obsah souboru */etc/sysconfig/network-scripts/ifcfg-eth0*
 - `DEVICE=eth0`
 - `ONBOOT=yes`
 - `TYPE=Ethernet`
 - `BRIDGE=ovzbridge`



■ Nastavení bridge na hostitelském systému.

- Vytvoření vlastního bridge **ovzbridge**. Obsah souboru */etc/sysconfig/network-scripts/ifcfg-ovzbridge*
 - DEVICE=ovzbridge
 - TYPE=Bridge
 - BOOTPROTO=static (dhcp)
 - ONBOOT=yes
 - IPADDR=10.10.21.70
 - NETMASK=255.255.255.192
 - DELAY=0
- **service network restart**
- **brctl show**



■ Diskové kvóty :

- ♦ OpenVZ rozlišuje dva typy diskových kvót :
 - **Kvóty první úrovně** umožňují omezit počet bloků a i-nodů pro jednotlivé VE.
 - **Kvóty druhé úrovně** umožňují správci daného VE nastavit limity pro jednotlivé uživatele a skupiny.
- ♦ Kvóty první úrovně jsou implicitně zapnuté (parametr **DISK_QUOTA** v globálním konfiguračním souboru).
- ♦ Oba typy kvót jsou na sobě nezávislé.



■ Diskové kvóty první úrovně :

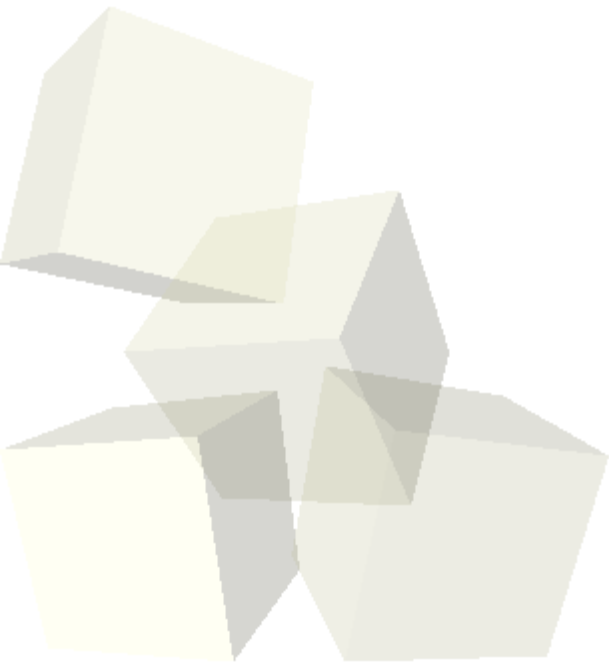
- ♦ Parametry pro nastavení kvót jsou umístěny v konfiguračním souboru daného VE, jednotkou je blok o velikosti 1024 B :
 - **diskspace** – maximální možný počet alokovaných bloků, má měkký a tvrdý limit.
 - **diskinodes** – maximální možný počet alokovaných i-nodů, má měkký a tvrdý limit.
 - **quotatime** – maximální možný čas překročení soft limitu v sekundách.
 - **quotauidlimit** – počet uživatelských/skupinových ID, kteří používají diskové kvóty. Pokud je nastaven na 0 jsou diskové kvóty druhé úrovně vypnuté.



■ Diskové kvóty první úrovně :

◆ Příklad nastavení kvót první úrovně :

- `vzctl set VE_ID --diskspace 10000:20000 --save`
- `vzctl set VE_ID --diskinodes 10000:20000 --save`
- `vzctl set VE_ID --quotatime 600 --save`





■ Diskové kvóty druhé úrovně :

- Hodnota **quotauidlimit** by měla být větší nebo rovna počtu uživatelů a počtu skupin.

→ `vzctl set VE_ID --quotauidlimit 100 --save`

- Administrátor jednotlivých VE pak používá standardní postupy pro nastavení kvót uživatelům.

- Zjištění nastavení kvót :

→ `vzquota stat VE_ID -t`



■ Plánovač procesů :

- ♦ OpenVZ umožňuje nastavit priority jednotlivých VE.
 - `vzctl set VEID --cpuunits 1000 --save`

- ♦ Příklad :

- `vzctl set 1001 --cpuunits 1000 --save`
- `vzctl set 1002 --cpuunits 2000 --save`
- `vzctl set 1003 --cpuunits 3000 --save`

Celkový počet jednotek $1000+2000+3000=6000$,
pak tedy mají jednotlivé VE plánovány tyto časy,
 **$VE1001=1/6=16\%$, $VE1002=1/3=34\%$,
 $VE1003=1/2=50\%$.**



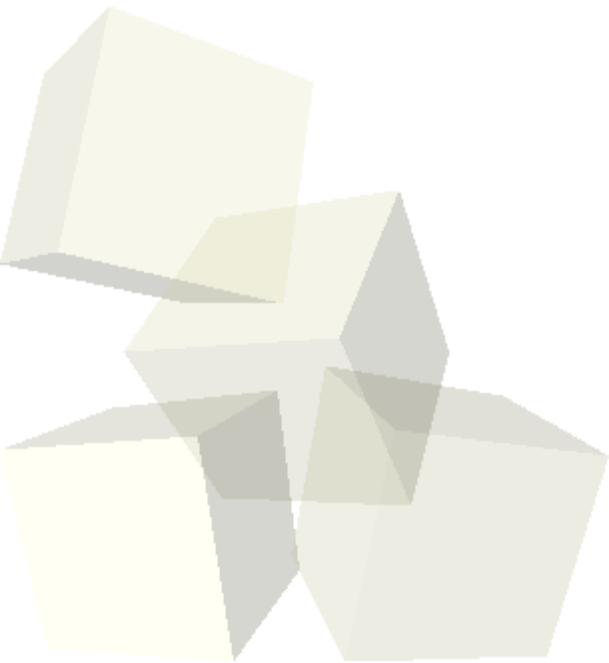
■ Zjišťování systémových informací :

- ♦ `cat /proc/user_beancounters`

Ize zjistit globálně v řídicím systému nebo individuálně v jednotlivých VE.

- ♦ `vzpid 78937`

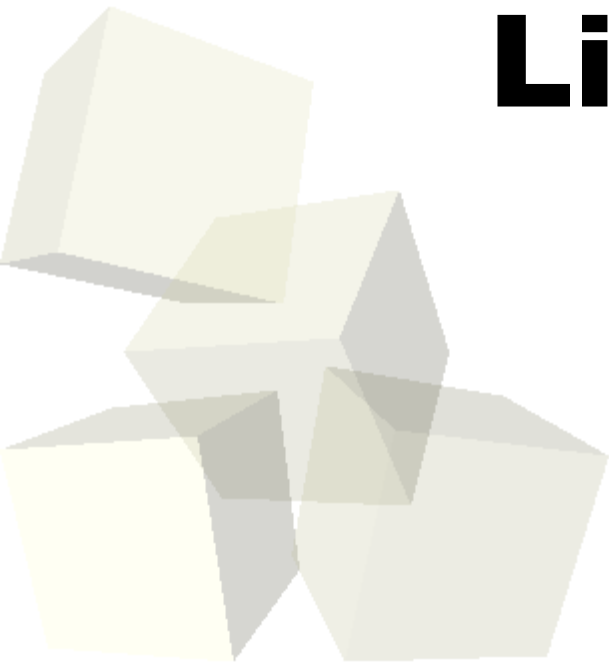
zjištění ve které VE_ID běží daný proces.





LXC

Linux Containers





■ Co je LXC ?

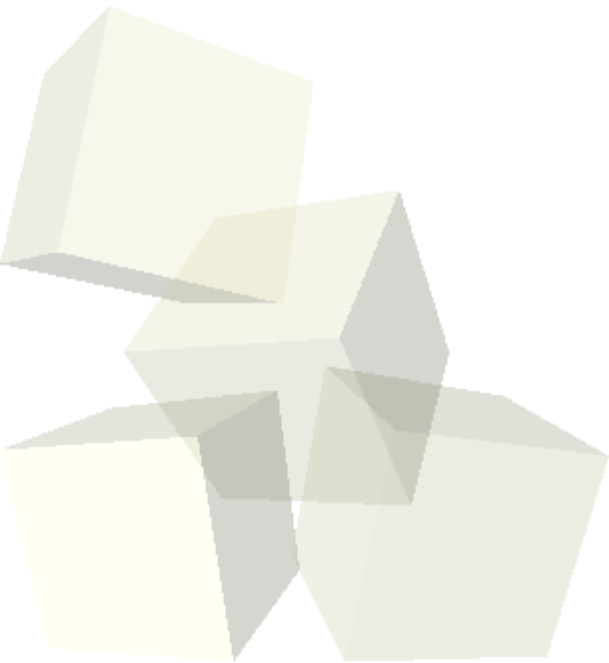
- ♦ Lehké „virtualizační“ řešení pro LINUX.
- ♦ Umožňuje vytvářet bezpečná a oddělená virtuální prostředí VE (virtual environment). V těchto VE běží jednotlivé, zcela separované instance OS.
- ♦ Ve VE je možné limitovat systémové a další zdroje.
- ♦ Jednotlivé VE se navzájem neovlivňují.

■ Podobná řešení :

- ♦ OpenVZ, VServer, FreeBSD Jail, Solaris Containers, ...



- V současné době je standardní součástí jádra.
- Management je možné provádět buď nativními ***lxc-...*** nástroji nebo pomocí ***libvirt***.





■ Instalace :

- ♦ yum install [epel-release](#)
- ♦ yum install [lxc](#) [lxc-extra](#)
- ♦ yum install [libvirt](#)
- ♦ yum install [lxc-templates](#) [debootstrap](#)

■ Spuštění komponent systému :

- ♦ systemctl enable [lxc](#)
- ♦ systemctl enable [libvirtd](#)

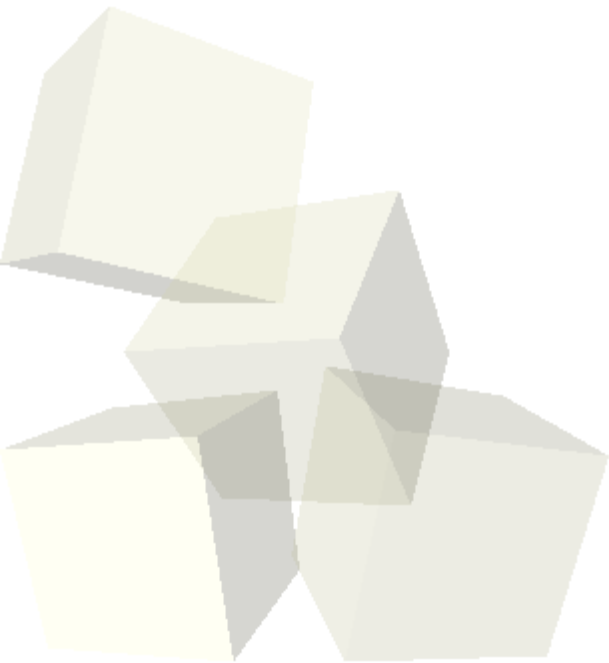
■ Otestování systému :

- ♦ [lxc-checkconfig](#)



■ Předpřipravené šablony :

- ♦ `/usr/share/lxc/templates`





■ Konfigurace :

- ♦ `/etc/lxc`

■ Domácí adresář LXC kontejnerů :

- ♦ `/var/lib/lxc`

■ Vytvoření nového kontejneru :

- ♦ `lxc-create -n nazev -t debian`
- ♦ `lxc-create -n nazev -t download`

■ Start kontejneru :

- ♦ `lxc-start -n nazev [-d]`



■ Získání terminálu v kontejneru :

- ♦ `lxc-console -n nazev -t 0`
- ♦ Opuštění terminálu **Ctrl+a q**

■ Výpis seznamu kontejnerů :

- ♦ `lxc-ls [-f] [--running]`

■ Informace o kontejneru :

- ♦ `lxc-info -n nazev`

■ Zastavení kontejneru :

- ♦ `lxc-stop -n nazev`



■ Online monitoring kontejneru :

- ♦ `lxc-monitor` -n nazev

■ Freezing kontejneru :

- ♦ `lxc-freeze` -n nazev
- ♦ `lxc-unfreeze` -n nazev

■ Klonování kontejneru :

- ♦ `lxc-clone` old new

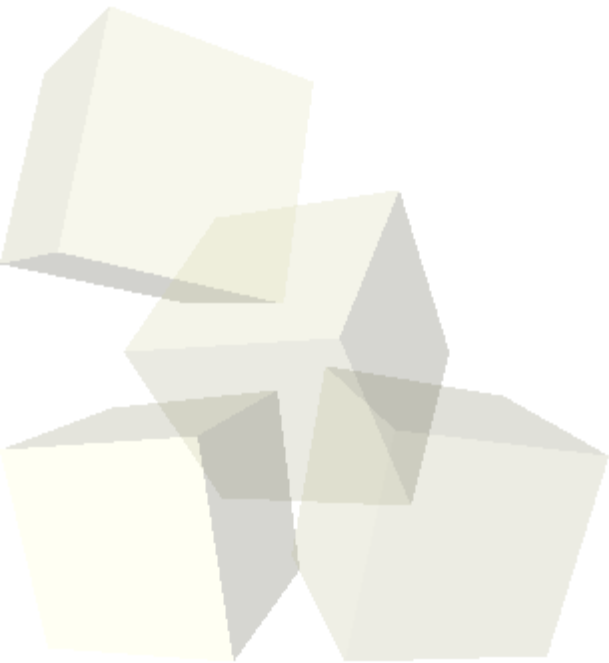
■ Smazání kontejneru :

- ♦ `lxc-destroy` -n nazev



■ Snapshoty :

- ♦ `lxc-snapshot -n nazev`
- ♦ `lxc-snapshot -L -n nazev`
- ♦ `/var/lib/lxc/snaps/`
- ♦ `lxc-snapshot -r snap0 -n nazev`



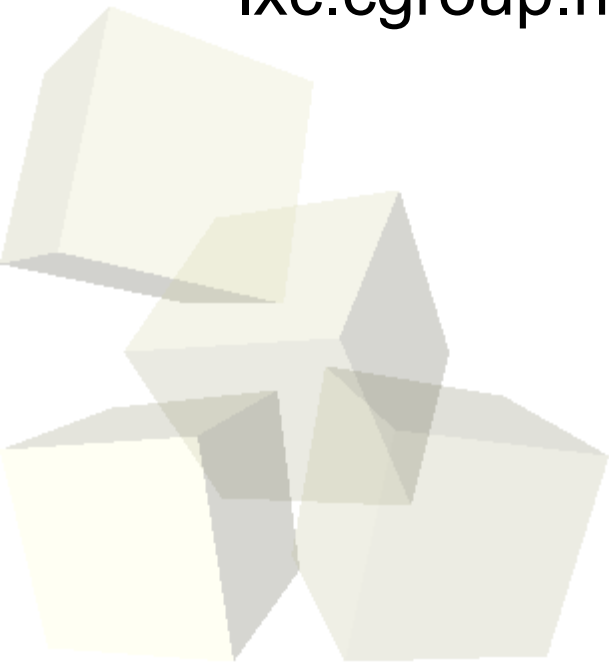


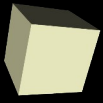
■ Nastavení parametrů pomocí *cgroups* :

- `cat /sys/fs/cgroup/memory/lxc/nazev/memory.usage_in_bytes`
- `cat /sys/fs/cgroup/memory/lxc/nazev/memory.limit_in_bytes`
- `lxc-cgroup -n nazev memory.limit_in_bytes 1G`

■ Nastavení parametrů pomocí *cgroups* v konfiguračním souboru :

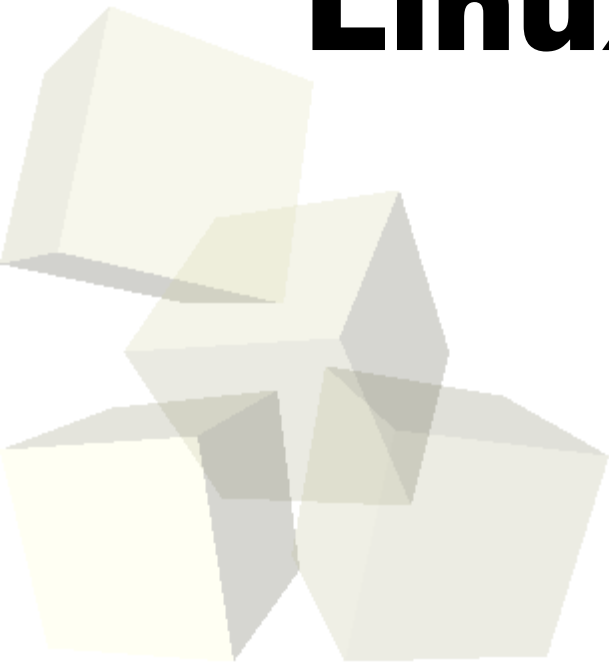
- `lxc.cgroup.memory.limit_in_bytes = 1G`





LXD

Linux Next generation Containers





■ Základní vlastnosti :

- **Secure by design** (unprivileged containers, resource restrictions and much more)
- **Scalable** (from containers on your laptop to thousand of compute nodes)
- **Intuitive** (simple, clear API and crisp command line experience)
- **Image based** (with a wide variety of Linux distributions published daily)
- **Support for Cross-host container and image transfer** (including migration with CRIU)
- **Advanced resource control** (cpu, memory, network I/O, block disk usage and kernel resources)
- **Device passthrough** (USB, GPU, unix character and block devices, disks and paths)
- **Network management** (bridge creation and configuration, cross-host tunnels, ...)
- **Storage management** (support for multiple storage backends, storage pools and storage volumes)



■ Instalace :

- ♦ **LXD** je doma na distribuci **Ubuntu**
- ♦ Na ostatních distribucích je vhodné instalovat LXD pomocí **snapu**.
- ♦ **Na RH systémech takto :**
 - **yum install epel-release**
 - **yum update**
 - **yum install snapd**
 - **systemctl enable snapd**
 - **systemctl start snapd**
 - **snap install lxd**



■ Instalace ZFS na RH systémech :

- ♦ `yum install https://zfsonlinux.org/epel/zfs-release-2-2$(rpm --eval "%{dist}").noarch.rpm`
- ♦ `[ yum install epel-release ]`
- ♦ `yum install kernel-devel`
- ♦ `yum install zfs`

■ Test ZFS:

- ♦ `zpool create storage1 /dev/sdb`



■ Kroky po instalaci :

- raději zakázat **SELINUX** (/etc/selinux/config)
- `grubby --args="user_namespace.enable=1" --update-kernel="$(grubby --default-kernel)"`
- `grubby --args="namespace.unpriv_enable=1" --update-kernel="$(grubby --default-kernel)"`
- `sh -c 'echo "user.max_user_namespaces=3883" > /etc/sysctl.d/99-usersns.conf'`
- **In -s /var/lib/snapd/snap /snap**
- uživatele s možností ovládání LXD přidat do skupiny **lxd**
- Vypnout **firewalld** (*systemctl disable firewalld*), případně i **libvirtd** (*systemctl disable libvirtd*).
- **lxd init**



■ Ovládání :

- ♦ terminálový nástroj pro ovládání - **lxc**
- ♦ **lxc remote list** ... výpis vzdálených serverů
- ♦ **lxc image list images:** ... výpis obsahu konkrétního repozitáře
- ♦ **lxc image list images: debian**

■ Vytvoření kontejneru / virtuálního stroje z image :

- ♦ **lxc init images:debian/11 debian1 [--vm]**
- ♦ **lxc launch images:debian/11 debian1 [--vm]**
- ♦ **lxc launch images:debian/11 debian1 < config.yaml**
- ♦ **lxc launch images:debian/11 debian1 --config limits.cpu=1
config limits.memory=192MiB**



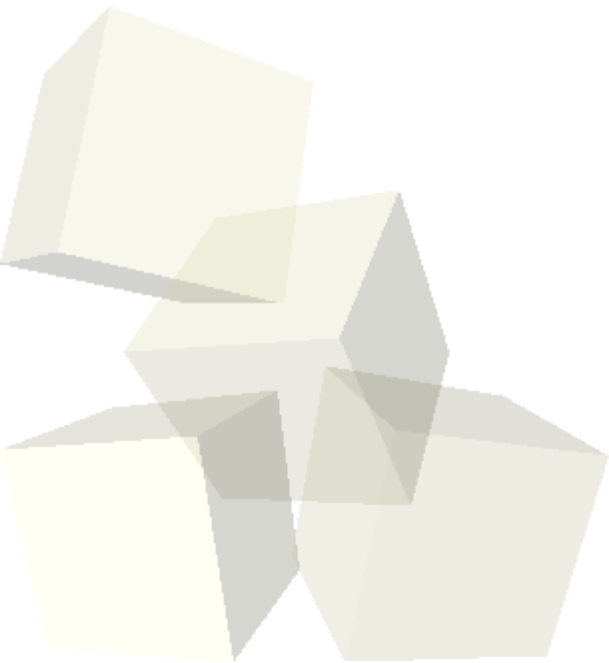
■ Ovládání kontejneru :

- ♦ **lxc list** [server:]
- ♦ **lxc start** debian1
- ♦ **lxc stop** debian1
- ♦ **lxc restart** debian1
- ♦ **lxc console** debian1
- ♦ **lxc shell** debian1
- ♦ **lxc exec** debian1 – apt update
- ♦ **lxc delete** debian1
- ♦ **lxc info** debian1



■ Kopírování souborů z/do kontejneru :

- **lxc file pull** container/path-in-container path-on-host
- **lxc file pull -r** container/path-in-container path-on-host
- **lxc file push** path-on-host container/path-in-container
- **lxc file push -r** path-on-host container/path-in-container



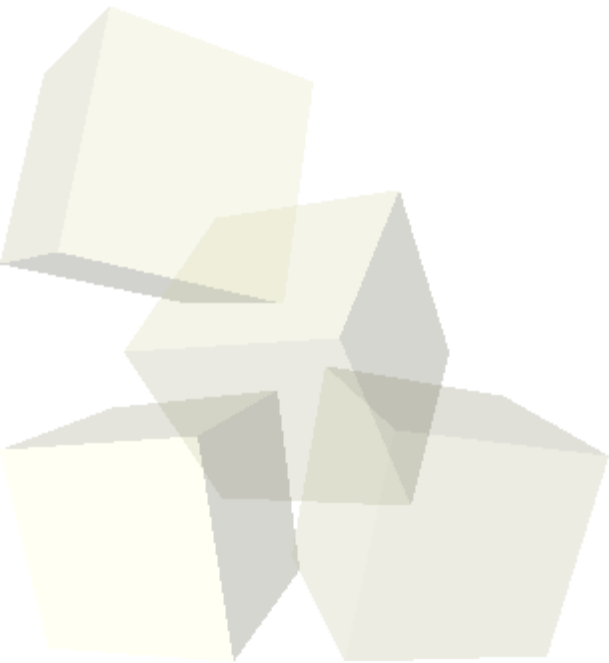


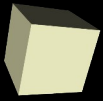
■ Manipulate s profily :

- ♦ **lxc profile list**
- ♦ **lxc profile show default**
- ♦ **lxc profile create profilename**
- ♦ **lxc profile edit profilename**
- ♦ **lxc profile set profilename key=value**
- ♦ **lxc profile list instancename**
- ♦ **lxc profile add instancename profilename**
- ♦ **lxc profile remove instancename profilename**



- Manipulace s konfigurací kontejneru :
 - ♦ `lxc config show instancename -e`
 - ♦ `lxc config edit instancename`
 - ♦ `lxc config set instancename key=value`





■ Snapshoty :

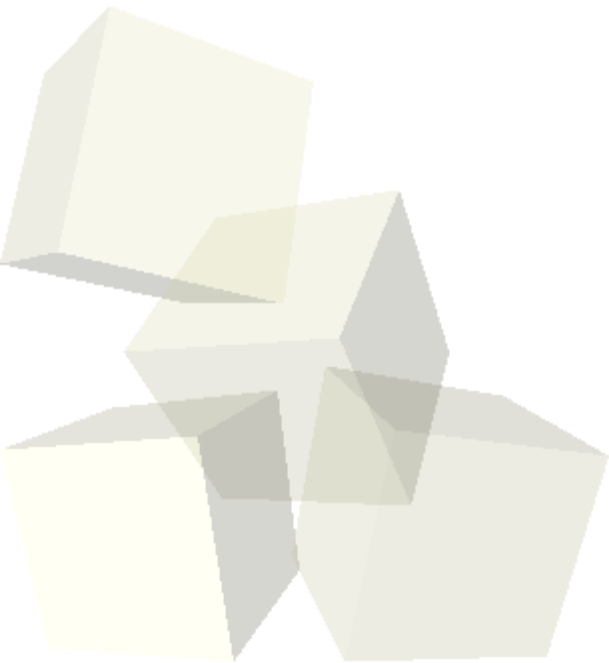
- ♦ **lxc snapshot container snapshot-name**
- ♦ **lxc restore container snapshot-name**
- ♦ **lxc delete container/snapshot-name**
- ♦ **lxc list container**
- ♦ **lxc info container snapshot-name**





■ Export / import :

- ♦ `lxc export container container.tar.xz`
- ♦ `lxc import container.tar.xz`





■ Kopírování kontejnerů :

- ♦ `lxc copy container1 container2`

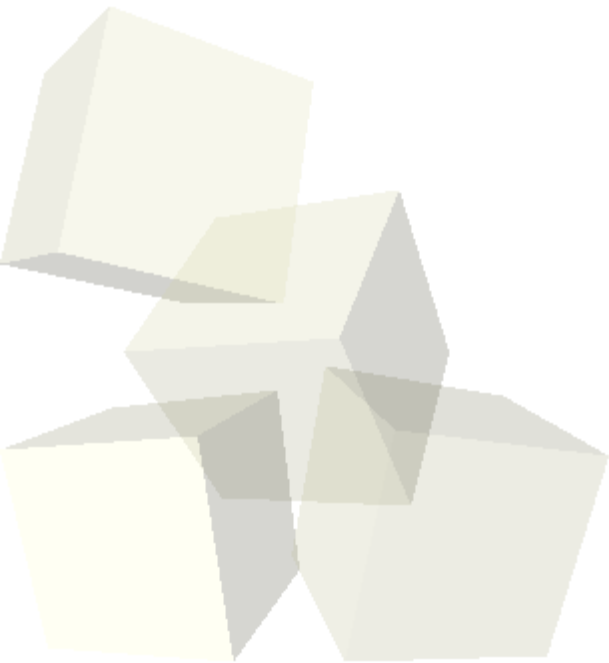
■ Nastavení LXD daemonu pro vzdálené ovládaní :

- ♦ `lxc config set core.https_address 0.0.0.0:8443`
- ♦ `lxc config set core.trust_password heslo`
- ♦ `lxc remote add myserver hostname_or_IP --password heslo`
- ♦ `lxc remote list`
- ♦ `snap set lxd criu.enable=true` (live migrate)
- ♦ `systemctl reload snap.lxd.daemon`



■ Migrace/kopírování kontejnerů :

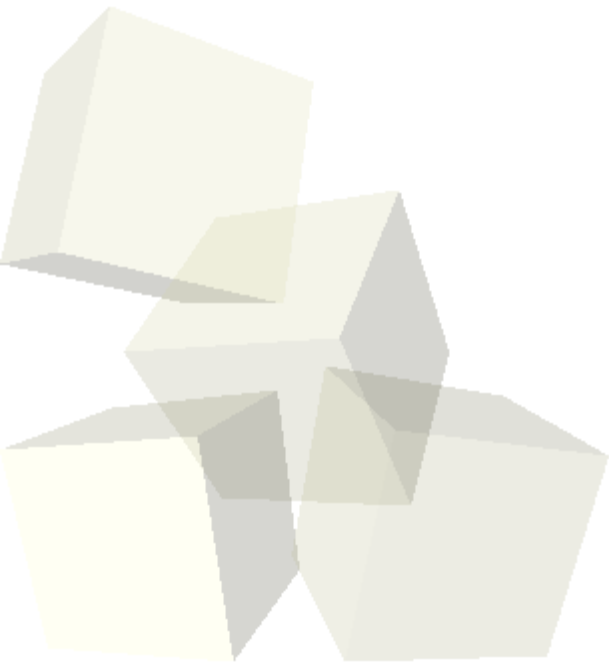
- ♦ **lxc move container server2:container [--verbose]**
- ♦ **lxc copy container server2:container**





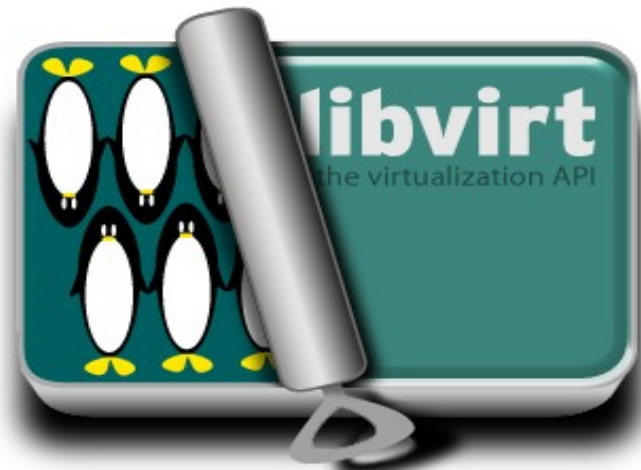
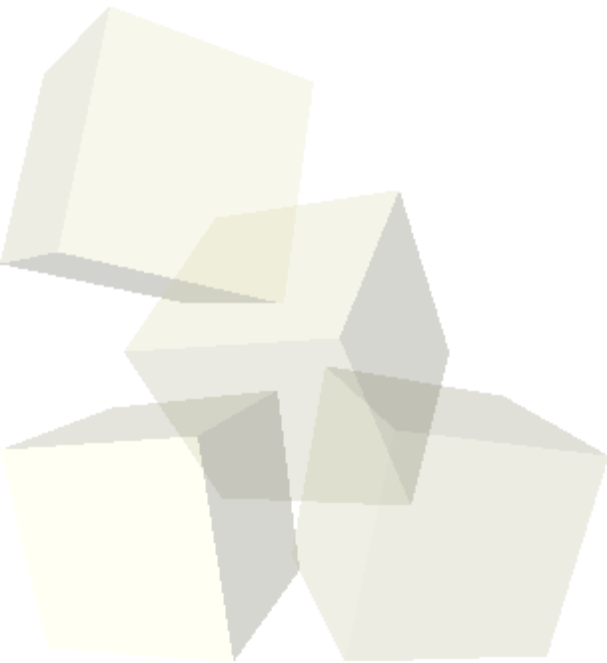
■ Nastavení limitů :

- ♦ `lxc config set container limits.memory 512MB`
- ♦ `lxc config set container limits.memory.swap false`
- ♦ `lxc config set container limits.cpu 2`
- ♦ `lxc config device override container root size=10GB`





libvirt Virtualization API





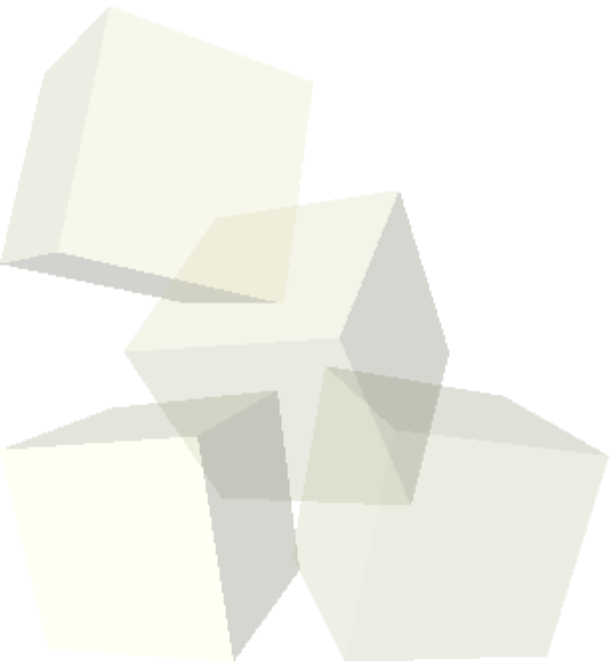
- Každý virtualizační systém jako KVM, XEN, OpenVZ disponuje svým vlastním rozhraním API, nástroji pro správu, atd.
- Je těžké vytvořit administrační a další nástroje pro ovládání těchto systémů.
- Knihovna **libvirt** poskytuje jednotné rozhraní pro různé virtualizační technologie :
 - ♦ Jednotné API v C (k dispozici i pro Python, Ruby, ...)
 - ♦ Konzolové rozhraní pro ovládání virtuálních strojů – **virsh**
 - ♦ V současnosti podporuje XEN, KVM/Qemu, OpenVZ, LXC, VirtualBOX, VMware ESX, MS Hyper-V, ...
 - ♦ Umožňuje komunikovat i vzdáleně po síti.



- Díky API je nad **libvirt** postaveno mnoho projektů, které je možné používat pro správu virtuálních strojů :
 - ♦ **virt-manager** ... grafické rozhraní pro instalaci a správu virtuálních strojů.
 - ♦ **virt-install** ... sada skriptů pro snadnou instalaci virtuálních strojů
 - ♦ **virt-clone** ... nástroj pro snadné klonování virtuálních strojů
 - ♦ a další



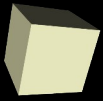
- `virt-install -n fedora9 -r 512 --vcpus 1 -f /var/lib/xen/fedora9.img -s 6 --vnc -p -l ftp://ftp.fi.muni.cz/pub/linux/fedora/linux/releases/9/Fedora/i386/os`





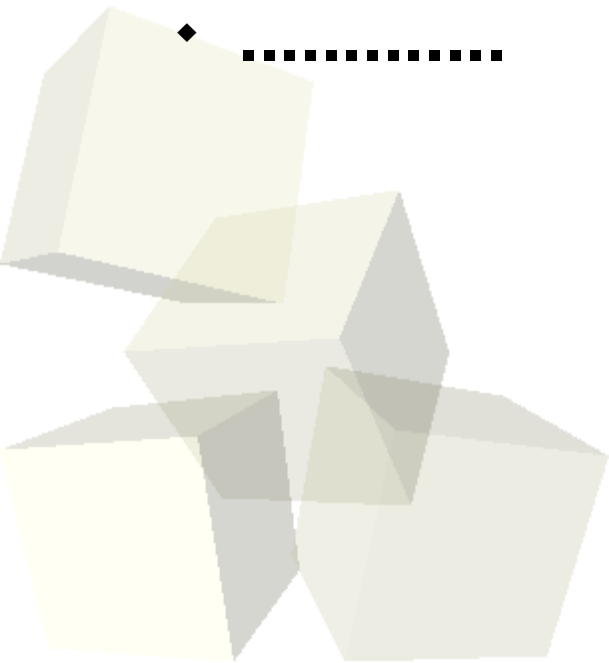
- **virsh** je základní terminálový nástroj pro management virtuálních strojů.
- Spojení s příslušným virtuálním strojem je popsáno pomocí libvirt identifikátoru URI. Příklad pro připojení ke XEN hypervisoru :

Local URI	Remote URI	Meaning
xen:///	xen://oírase/	Connect to the Xen hypervisor running on host oírase using TLS.
NULL	remote://oírase/	Connect to the "default" hypervisor running on host oírase using TLS.
xen:///	xen+ssh://oírase/	Connect to the Xen hypervisor running on host oírase by going over an ssh connection.
test:///default	test+tcp://oírase/default	Connect to the test driver on host oírase using an unsecured TCP connection.



■ Základní příkazy :

- ♦ virsh **start** VM
- ♦ virsh **stop** VM
- ♦ virsh **list**
- ♦ virsh **suspend** VM
- ♦ virsh **resume** VM
- ♦ virsh **dominfo** VM
- ♦



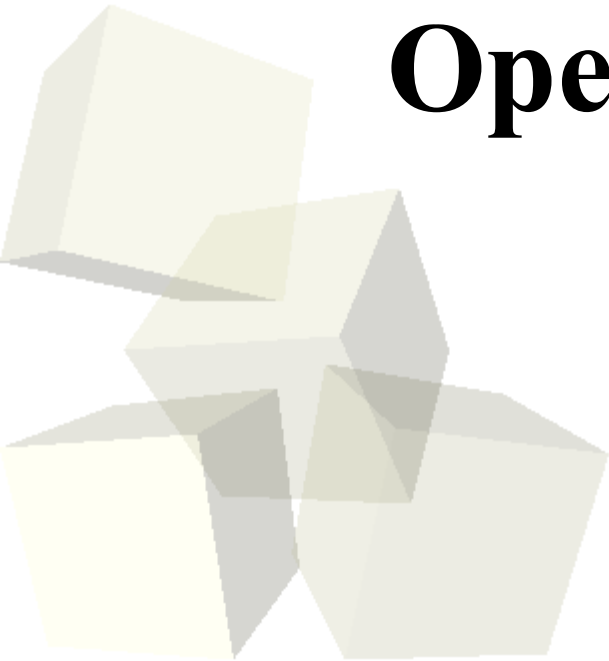


■ Editace a transformace konfiguračních formátů :

- ♦ **virsh edit VM** ... editace XML konfigurace VM.
- ♦ **virsh dumpxml VM** ... export XML konfigurace VM.
- ♦ **virsh domxml-to-native format config.xml** ...
konverze XML konfigurace do nativního formátu
konkrétního virtualizéru.
(format : **xen-xl**, **qemu-argv**, ...).
- ♦ **virsh define config.xml** ... import XML konfigurace.



Open Source Virtualizer

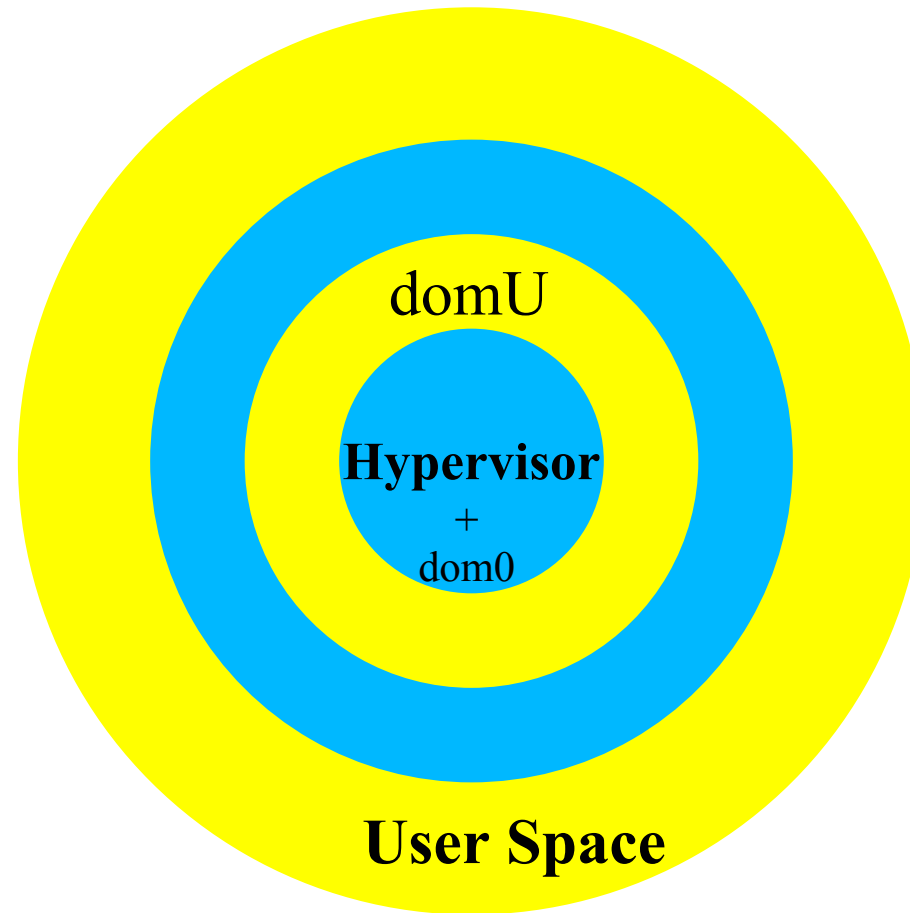




- **Xen** je volně šířený **hypervisor** (virtual machine monitor) pro architektury **IA-32**, **x86**, **x86-64**, **IA-64**, **PowerPC 970** a **ARM**.
- Umožňuje běh více operačních systémů.
- **Xen** se skládá z **hypervisoru**, který je provozován ve vrstvě **ring-0** procesoru třídy Intel a z několika hostovaných operačních systémů (domén).
- První se nazývá **dom-0** a hypervisor ho používá k přístupu k fyzickému hardware počítače.



- V **dom-0** je možné používat tyto modifikované operační systémy : **LINUX**, **NetBSD** a **Solaris**.
- Administrátor systému využívá **dom-0** pro řízení a další obsluhu celého virtualizačního systému.
- Vlastní hostované operační systémy se nazývají **dom-U**. V **paravirtualizovaném režimu** je možné provozovat jen **modifikované** operační systémy.
- V **plně virtualizovaném** prostředí je možné provozovat nemodifikované OS. Xen podporuje virtualizační rozšíření **AMD-V** (**Pacifica**) a **Intel VT** (**VT-x** Vanderpool, **VT-i** Silverdale).



**Schéma úrovní CPU při použití
virtualizéru Xen bez virtualizačního rozšíření**

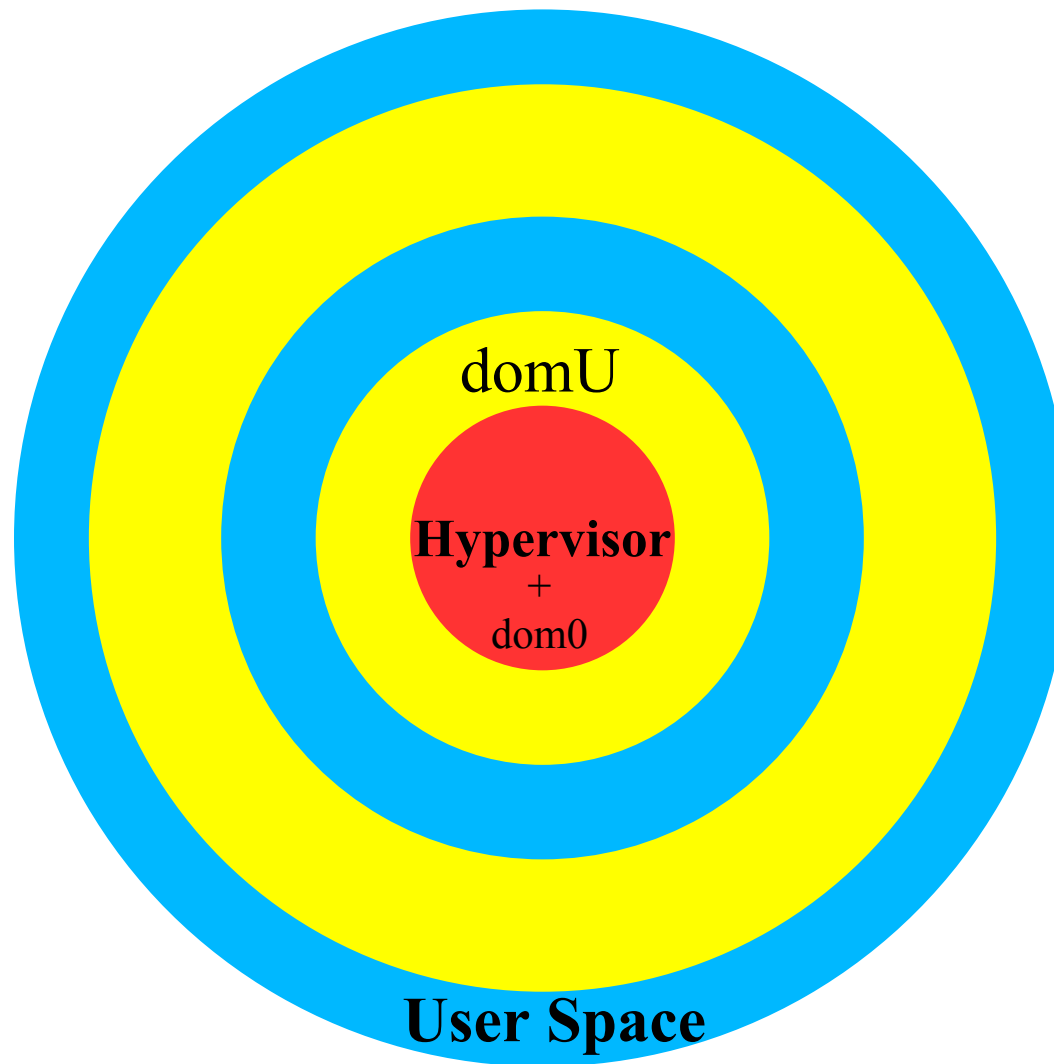
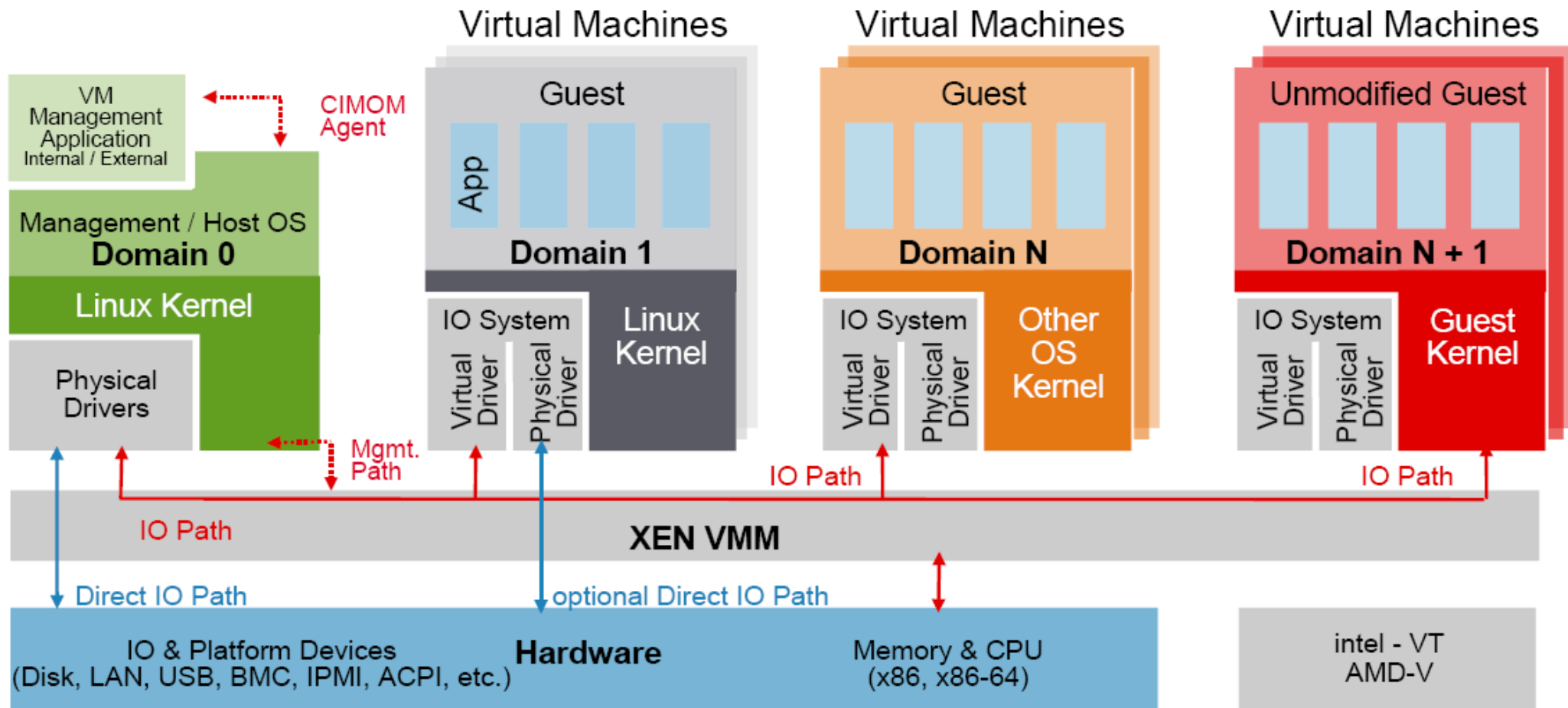


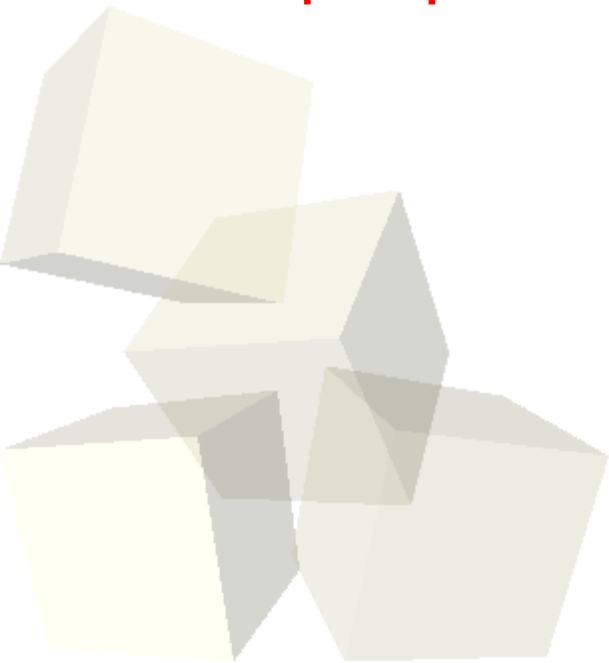
Schéma úrovní CPU při použití virtualizéru Xen s virtualizačním rozšířením





- Virtualizační podporu v procesoru lze zjistit z příznaků daného CPU : *cat /proc/cpuinfo*
 - ♦ příznak **vmx** - technologie Intel
 - ♦ příznak **svm** - technologie AMD

Pozn.: V praxi je nutná i podpora v BIOSu a virtualizaci musí podporovat i chipset !!!





■ Podpora IOMMU (PCI passthrough) :

- **Intel** : `dmesg | grep -e DMAR -e IOMMU`
- **AMD** : `dmesg | grep -e IOMMU`

1) Je nutné to zapnout v BIOSu

2) Upravit nastavení **GRUBu** :

```
/etc/default/grub : GRUB_CMDLINE_LINUX="intel_iommu=on"  
(amd_iommu=on)  
update-grub && reboot
```



■ Požadavky na systém :

- Procesor třídy **P6 nebo lepší**
- Standardní LINUX-like operační systém se zavaděčem **GRUB**
- Instalované balíky : ***iproute2***, ***bridge-utils***, ***udev*** subsystem

■ Požadavky na systém v případě instalace ze zdrojových kódů :

- Vývojové nástroje (***GCC***, ***binutils***, ***GNU Make***, atd.)
- ***zlib-dev***
- ***python-dev***
- ***openssl-dev***
- ***libncurses-dev***
- ***xorg-dev***



■ Instalace :

♦ Ze zdrojových kódů :

- <http://www.xenproject.org/>
- nutné zkompilovat a nainstalovat vlastní **hypervisor**, **modifikované jádro pro dom0** a další **administrátorské a obslužné nástroje** (**xend**, **xm**, atd.)
- postup viz originální dokumentace

♦ Z balíčků dané distribuce :

- drtivá většina dnešních distribucí LINUXu obsahuje **kompletní podporu pro Xen** a další rozšiřující nástroje jako je např. aplikace **virt-manager** firmy RedHat, atd.



■ Instalace v CentOS 7:

- ♦ Centos 7 obsahuje oficiální podporu XEN.
- ♦ Stačí přidat XEN repozitář :

```
yum install centos-release-xen  
yum update
```

- ♦ Dále je třeba instalovat metabalíček **xen**, případně další pomocné balíky : **libvirt**, **virt-manager**, **libvirt-daemon-driver-xen**, **libvirt-daemon-xen**, případně další.
- ♦ Posledním krokem je konfigurace zavaděče :

```
grub-bootxen.sh
```



■ Konfigurace zavaděče **GRUB** (*grub.conf*) :

- ♦ title Xen 3.0.3-1-amd64
- root (hd0,0)
- kernel /xen-3.0.3-1-amd64.gz dom0_mem=512M
- module /vmlinuz-2.6.18-5-xen-amd64 root=/dev/hda1 ro console=tty0
- module /initrd.img-2.6.18-5-xen-amd64

■ V **dom0** je třeba zajistit start démona **xend**, pokud chceme používat administrační nástroj **xm**. (*xm* je od verze 4.5 zcela odstraněn a nahrazen **xl**).

■ Nástroj **xl** je s původním **xm** co do ovládání kompatibilní.



■ Konfigurace hostovaných systémů (*domU*) :

/etc/xen/hippo.cfg

- ♦ kernel = '/boot/vmlinuz-2.6.18-5-xen-vserver-amd64'
ramdisk = '/boot/initrd.img-2.6.18-5-xen-vserver-amd64'
memory = '3000'
vcpus = '3'

root = '/dev/sda1 ro'

disk = ['phy:xen/hippo-disk,sda1,w', 'phy:xen/hippo-swap,sda2,w',
'phy:hda2,sdb1,w', 'phy:hda3,sdb2,w', 'phy:hda4,sdb3,w', 'phy:hdb1,sdc1,w',
'phy:sdc1,sdd1,w']

name = 'hippo'

vif = ['ip=147.32.193.32, mac=00:D0:B7:B8:1F:C2']

on_poweroff = 'destroy'

on_reboot = 'restart'

on_crash = 'restart'



■ Řídící nástroj *xm* [*xl*] : *xm* <příkaz> [argumenty]

console	Attach to <Domain>'s console.
create	Create a domain based on <ConfigFile>.
destroy	Terminate a domain immediately.
dump-core	Dump core for a specific domain.
help	Display this message.
list	List information about all/some domains.
mem-set	Set the current memory usage for a domain.
migrate	Migrate a domain to another machine.
pause	Pause execution of a domain.
reboot	Reboot a domain.
restore	Restore a domain from a saved state.
save	Save a domain state to restore later.
shutdown	Shutdown a domain.
top	Monitor a host and the domains in real time.
unpause	Unpause a paused domain.
uptime	Print uptime for a domain.
vcpu-set	Set the number of active VCPUs for allowed for the domain.

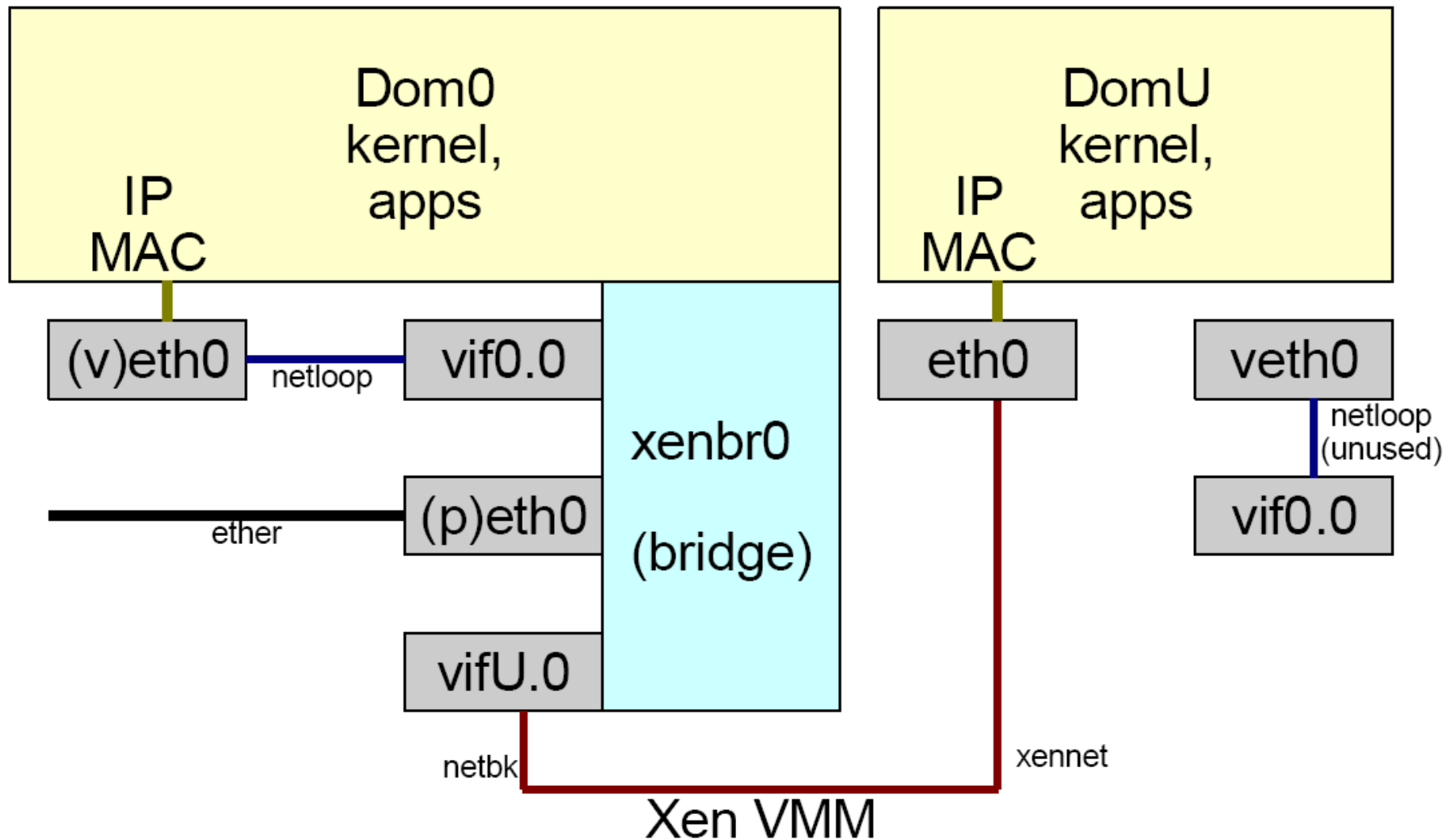


■ Bloková zřízení hostovaných systémů :

- ♦ **Fyzické disky a zařízení (VBD)**
 - `disk=['phy:hda2,sda1,w']`
- ♦ **Image v souboru (file backed VBD)**
 - `disk=['file:/path/to/vm1disk,sda1,w']`
`dd if=/dev/zero of=vm1disk bs=1k count=1`
`mkfs -t ext3 vm1disk`
- ♦ **Použití LVM (LVM backed VBD)**
 - `disk=['phy:vg/myvmdisk1,sda1,w']`
`pvcreeate /dev/hda1`
`vgcreate vg /dev/hda1`
`lvcreate -L4096M -n myvmdisk1 vg`
`mkfs -t ext4 /dev/vg/myvmdisk1`



■ Sít'ová rozhraní hostovaných systémů :





■ PCI Passthrough :

♦ Zařízení PCI

Xen dokáže nabízet konkrétní PCI zařízení přímo do konkrétního hostovaného systému *domU*. V doméně *dom0* je třeba dané zařízení zakázat. PCI zařízení je identifikováno tzv. PCI adresou : **(bus:slot.func)**. Seznam zařízení je možné získat utilitou *lspci*. V doméně *dom0* je nutné upravit **zavaděč grub.conf**.

...root=/dev/sda4 ro pciback.hide=(bus:slot.func)

V konfiguračním souboru hostovaného systému :

```
pci=['01:00.0', '02:03.0']
```



■ PCI Passthrough (dynamické řešení) :

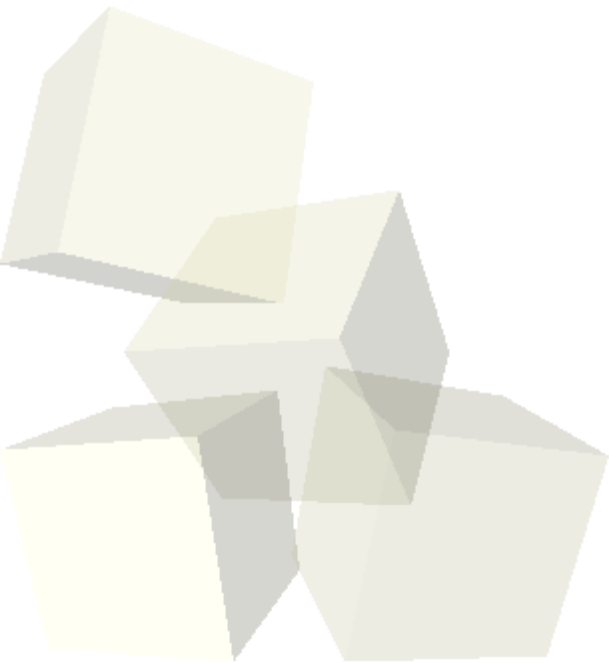
- ♦ **modprobe xen-pciback**
- ♦ xl **pci-assignable-list**
- ♦ xl **pci-assignable-add** 08:00.0
- ♦ xl **pci-attach** <domain-id> <pci device>
- ♦ xl **pci-detach** <domain-id> <pci device>



■ Řídící nástroj *xm* : *xm* <příkaz> [argumenty]

♦ *xm list*

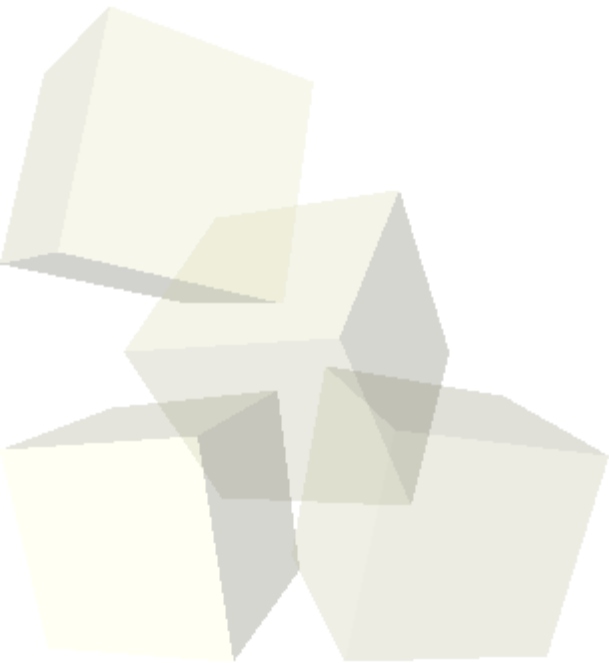
Name	ID	Mem(MiB)	VCPUs	State	Time(s)
Domain-0	0	512	1	r-----	427.1
asinus	3	2000	1	-b----	1.1
hippo	1	3000	3	-b----	887.9
octopus	2	2000	2	-b----	94.2





■ Uložení a obnovení domény *domU* :

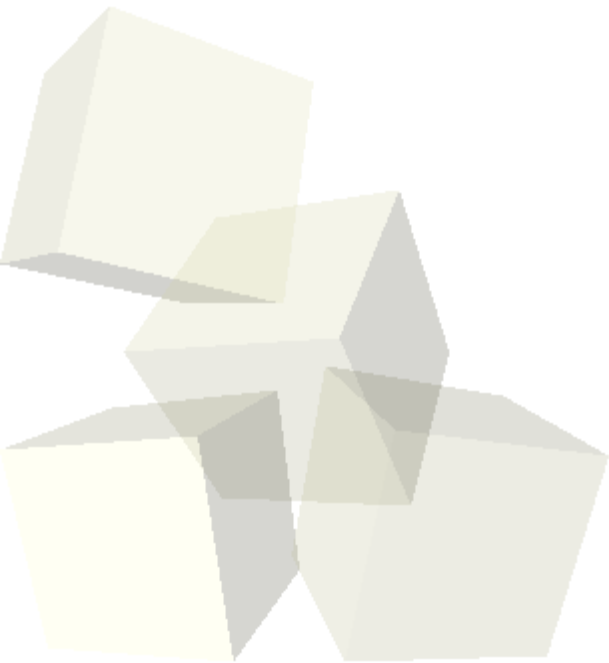
- ♦ **uložení a zastavení domény :**
 - `xm save hippo hippo.chk`
- ♦ **obnovení domény :**
 - `xm restore hippo.chk`





■ Migrate domén *domU* :

- ♦ migrate domény na jiný stroj :
 - `xm migrate [--live] hippo xen2.firma.cz`





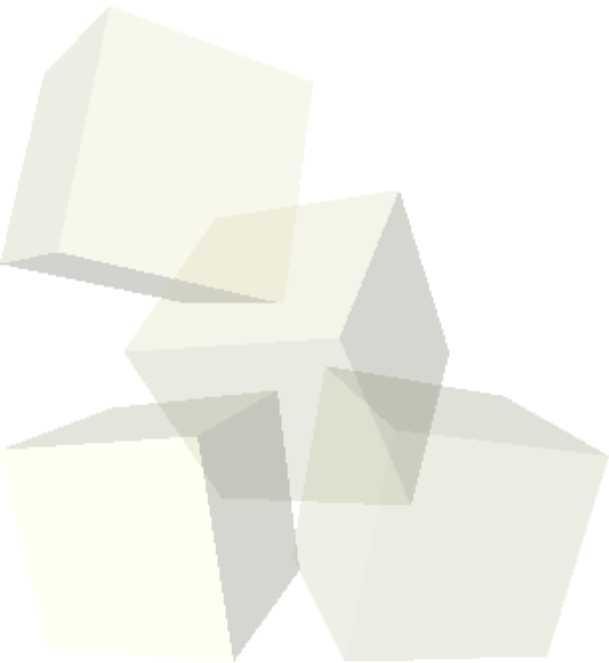
- **Sít'ové blokové zařízení NBD (*strana serveru*) :**
 - ♦ **Příprava blokového loopback zařízení :**

```
dd if=/dev/zero of=/var/nbd/nbd1.img bs=1024k count=512  
losetup /dev/loop0 /var/nbd/nbd1.img  
mkfs -t ext3 /dev/loop0  
mount -t ext3 /dev/loop0 /mnt/nbd1  
  
umount /mnt/nbd1  
losetup -d /dev/loop0
```
 - ♦ **Start *nbd* serveru :**

```
modprobe nbd  
nbd-server 25000 /var/nbd/nbd1.img
```



- **Sít'ové blokové zařízení NBD** (*strana klienta*) :
 - ♦ **Připojení sít'ového blokového zařízení :**
`modprobe nbd`
`nbd-client nbd.server.cz 25000 /dev/nbd0`
`mount /dev/nbd0 /mnt/nbd1`





■ Plná virtualizace (*provoz nemodifikovaných systémů*) :

- Konfigurační soubor domU (*winxp.cfg*) :

.....

kernel = '/usr/lib/xen/boot/hvmloader'

builder = 'hvm'

memory = '256'

device_model='/usr/lib/xen/bin/qemu-dm'

.....

disk = ['file:/xen/domains/windisk.img,ioemu:hda,w',
'file:/xen/winimage.iso,ioemu:hdc:cdrom,r']

name = 'win01'

vif = ['type=ioemu']

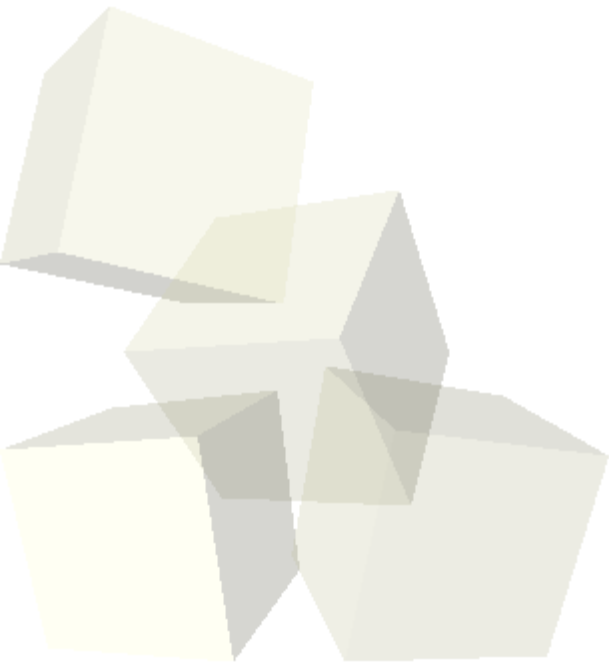
boot='d'



QEMU

Open source processor emulator

<http://www.qemu.org/>





- **QEMU** je emulátor procesoru používající dynamický překlad kódu. Pracuje ve dvou režimech :
 - ♦ **Plná emulace** (*full system emulation*) – QEMU emuluje celý systém (např. PC) včetně jednoho nebo několika procesorů a různých periférií. Umožňuje nastartovat a provozovat přímo jiný operační systém.
 - Emuluje několik HW platforem, např. : x86, x86-64, Alpha, ETRAX CRIS, MIPS, SPARC, ARM, atd.
 - ♦ **Uživatelská emulace** (*user mode emulation*) – Umožňuje spustit jeden proces, který je určený pro odlišnou platformu (cross-compilation, cross-debugging).



■ Na platformě PC QEMU simuluje tento hardware :

- i440FX host PCI bridge and PIIX3 PCI to ISA bridge
- Cirrus CLGD 5446 PCI VGA card or dummy VGA card with Bochs VESA extensions (hardware level, including all non standard modes).
- PS/2 mouse and keyboard
- 2 PCI IDE interfaces with hard disk and CD-ROM support
- Floppy disk
- PCI/ISA PCI network adapters
- Serial ports
- Creative SoundBlaster 16 sound card
- ENSONIQ AudioPCI ES1370 sound card
- Adlib(OPL2) - Yamaha YM3812 compatible chip
- PCI UHCI USB controller and a virtual USB hub



■ Základní použití plně emulovaného módu :

qemu [options] [disk_image]

Základní parametry :

- ♦ **-M *machine*** typ emulovaného stroje
- ♦ **-hd[abcd] *file*** hard disk image
- ♦ **-cdrom *file*** CD-ROM image
- ♦ **-boot [a|c|d|n]** boot médium (floppy=a, HD=c, CD-ROM=d, etherboot=n)
- ♦ **-m *size*** velikost RAM v megabytech
- ♦ **-soundhw *card*** povolit audio
- ♦ **-localtime** nastavit čas podle lokálníhoho



- Základní použití plně emulovaného módu :

qemu [options] [disk_image]

Parametry zobrazení :

- ♦ **-nographic** vypne grafický výstup
- ♦ **-no-frame** vypne dekorace SDL okna
- ♦ **-full-screen** nastartuje v režimu *full-screen*
- ♦ **-k language** nastavení mapy klávesnice
- ♦ **-vnc display** použití VNC (viz dokumentace)
- ♦ **-daemonize** spuštění na pozadí

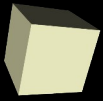


- Základní použití plně emulovaného módu (síťové parametry) :

qemu [options] [disk_image]

Síťové parametry :

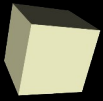
- ♦ **-net nic** vytvoří nové síťové rozhraní v hostovaném systému
- ♦ **-net user** použije jednoduché připojení hostovaného systému do sítě (použije NAT, Qemu simuluje DHCP)
- ♦ **-net tap** spojí rozhraní v hostovaném systému s jedním **tap** rozhraním v hostitelském systému.



■ Síťové parametry – příklady :

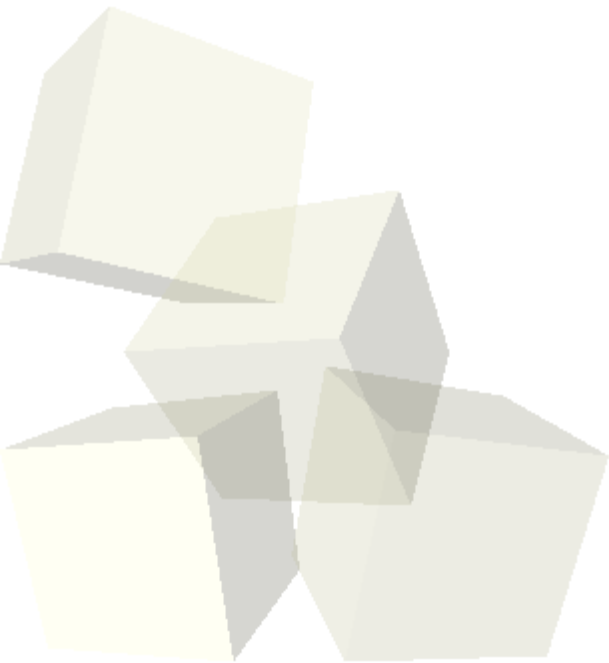
- ♦ `-net nic -net user`
- ♦ `-net nic -net tap`
- ♦ `-net nic,vlan=0 -net tap,vlan=0,ifname=tap0`
`-net nic,vlan=1 -net tap,vlan=1,ifname=tap1`
- ♦ `-net nic,vlan=0 -net tap,vlan=0,ifname=tap0`
`-net nic,vlan=1 -net user,vlan=1`

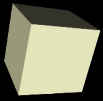




■ Ovládání (*v grafickém režimu*) :

- ♦ Ctrl-Alt-f režim full-screen
- ♦ Ctrl-Alt-1 monitor virtualizovaného systému
- ♦ Ctrl-Alt-2 QEMU monitor
- ♦ Ctrl-Alt-3 sériový port
- ♦ Ctrl-Alt přepnutí ovládaní hostovaného nebo hostujícího systému





- **QEMU monitor** umožňuje komplexní ovládání běžícího hostovaného systému :
 - ♦ Připojování / odpojování výměnných médií jako jsou diskety a CD-ROM nosiče.
 - ♦ Uložení a znovuoobnovení virtuálního stroje.
 - ♦ Ladění a profilování hostovaného systému.





■ Základní příkazy QEMU monitoru :

- ♦ *info subcommand*
 - *network* informace o síťovém nastavení
 - *block* zobrazí bloková zařízení
 - *pci* zobrazí emulovaná PCI zařízení
 - *usb* zobrazí USB zařízení zapojená do virtuálního USB hubu
 - *usbhost* zobrazí všechna hostovaná USB zařízení
- ♦ *eject [-f] device* vysune výměnné médium
- ♦ *change device setting*



■ Diskové obrazy (*disk images*) příkaz *qemu-img* :

qemu-img command [command options]

- ♦ `qemu-img create [-e] [-z] [-b base_image] [-f fmt] filename [size]`
 - **filename** jméno souboru s obrazem
 - **-f** formát obrazu (**raw**, **qcow2**, qcow, **vmdk**, ...)
 - **size** velikost obrazu v kilobytech, lze použít přípony M (megabyty), G (gigabyty)
 - **-e** výsledný obraz bude šifrován (qcow formát)
 - **-z** výsledný obraz bude komprimován (qcow formát)



■ Diskové obrazy (*disk images*), typy obrazů :

- ♦ **raw** ... výchozí a jednoduchý formát, umožňují snadnou konverzi do jiných formátů.
- ♦ **qcow2** ... nativní formát QEMU, umožňuje *zlib* kompresi, *AES* šifrování a *několikanásobné VM snapshots*.
- ♦ **vmdk** ... formát kompatibilní s VMWare 3 a 4.



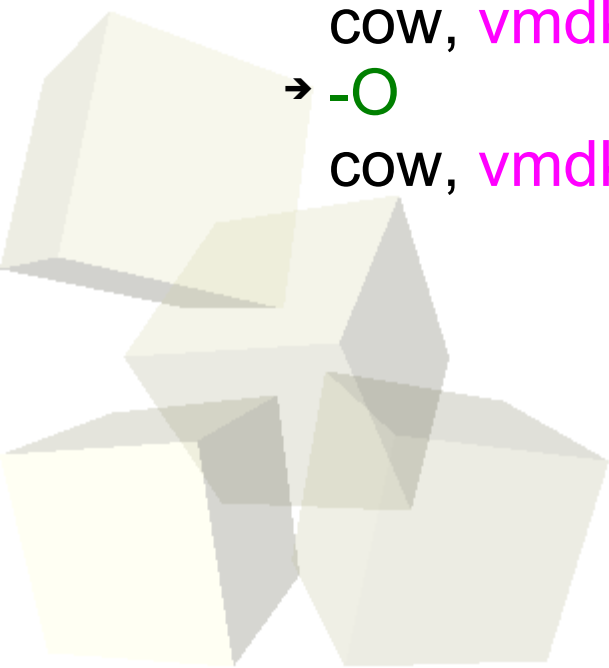


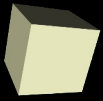
■ Konverze diskových obrazů :

qemu-img command [command options]

- ♦ `qemu-img convert -f input format -O output format
input.img output.qcow2`

- `-f` formát vstupního obrazu (`raw`, `qcow2`,
`cow`, `vmdk`, ...)
- `-O` formát výstupního obrazu (`raw`, `qcow2`,
`cow`, `vmdk`, ...)

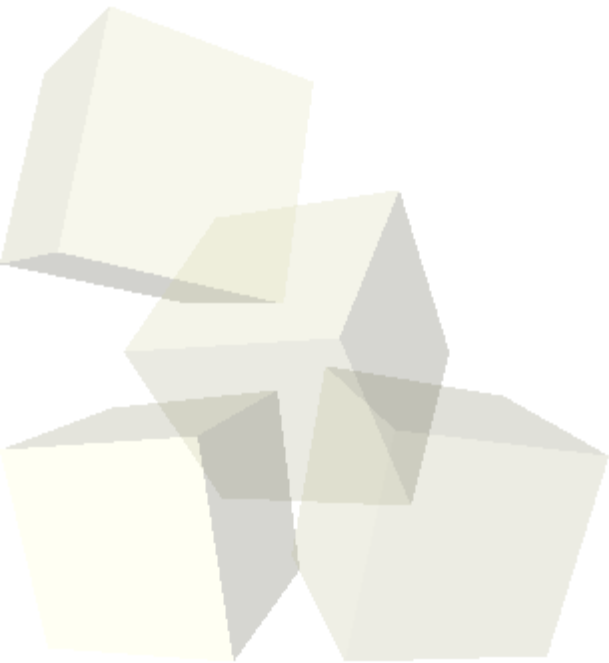




■ Změna velikosti diskových obrazů :

qemu-img command [command options]

- ♦ `qemu-img resize centos6-template.qcow2 +300GB`





■ Připojení diskových obrazů (*qemu-nbd*) :

- ♦ modprobe nbd
- ♦ **qemu-nbd --format=qcow2 --connect=/dev/nbd0 disk.qcow2**
- ♦ fdisk /dev/nbd0
- ♦ partx -a /dev/nbd0
- ♦ mount -t **ext4** /dev/nbd0**p1** /mnt/disk
- ♦ **qemu-nbd --disconnect /dev/nbd0**



■ Připojení diskových obrazů (*libguestfs*) :

- ♦ **guestmount -a disk.qcow2 -m /dev/sdx1 /mnt**

libguestfs: error: mount_options: mount_options_stub:
/dev/sdx1: No such file or directory

guestmount: '/dev/sdx1' could not be mounted.

guestmount: Did you mean to mount one of these
filesystems?

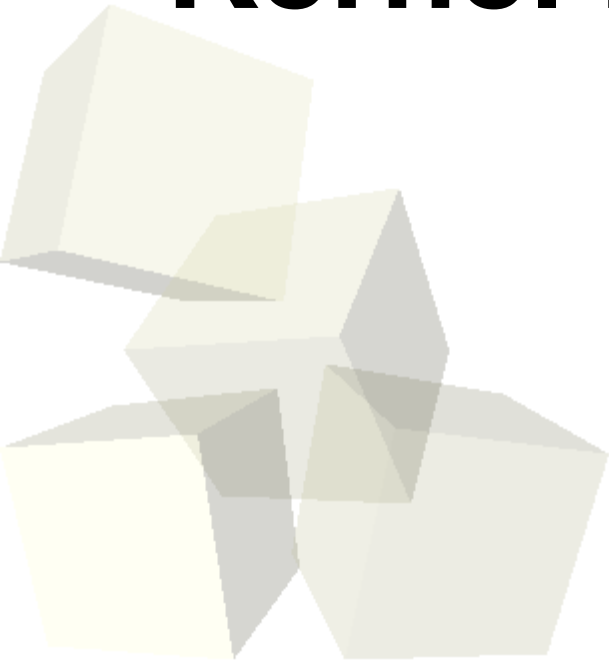
guestmount: /dev/sda1 (ext4)

guestmount: /dev/VolGroup00/LogVol00 (ext4)

guestmount: /dev/VolGroup00/LogVol01 (swap)



Kernel based Virtual Machine



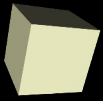


- **KVM – Kernel-based Virtual Machine** je volně šířený **hypervisor** (virtual machine monitor) pro architektury **x86** a **x86-64**.
- **KVM** je přímou součástí **LINUX** jádra (v současné době je implementováno jako modul). KVM je portován také na FreeBSD.
- **KVM** vyžaduje hardwarovou podporu virtualizace **Intel VT** nebo **AMD-V**.





- KVM se skládá ze dvou jaderných modulů :
 - ♦ **kvm.ko**
 - ♦ **kvm_intel.ko** nebo **kvm_amd.ko**
- Základním administračním nástrojem pro KVM je modifikovaný nástroj **qemu**.
- KVM dnes hlavně vyvíjí RedHat a je základem jejich **RHV**.
- V RHEL je KVM standardně obsaženo a pro administraci se používá rozhraní **libvirt**.



- **KVM** používá **QEMU** jako svou HW simulační vrstvu.
- KVM obsahuje vlastní paravrstvu jménem **VirtIO**.
- Linux a některé další systémy **VirtIO** nativně podporují, pro další OS jako Windows lze paraovladače doinstalovat.

